A Remotely Accessible Co-Simulation Testbed for Synchrophasor Monitoring and Control Under Cyber Attack

Md Mazharul Islam, Sunjare Zulfiker, Masum Reza, and Hafiz Abdur Rahman

***Abstract*—Control centers use phasor measurement units (PMUs) to correct grid problems within a second, but this depends on the network, the phasor data concentrator (PDC) and whether the frames can be trusted. Testbeds for this chain need costly real-time hardware, and open-source co-simulations target distribution systems or omit the measurement chain. This paper presents an open-source, remotely accessible co-simulation testbed that closes the synchrophasor loop at transmission level. GridPACK runs the grid on a computing cluster, NS-3 carries every PMU frame and command through a PDC, and a control center performs state estimation, chi-square bad data detection with PMU removal, and voltage control, all synchronized by HELICS and shared among users through a web dashboard. On the IEEE 118-bus system, 1087 runs covered five fault types at 22 locations, four attack types, two PMU placements, network sweeps and detector tests. Delaying or dropping the frames of PMUs observing the faulted bus hid the fault at all 18 informative locations with 32 PMUs and at 16 with 68, failing only where the fault also showed at a bus other PMUs observed. A detected injection still hid the fault at three locations, because removing the suspect PMU left the bus unobservable. Without any attack, a 40 ms mean latency with a 20 ms wait window hid a fault in six of ten seeds. Watching the neighbourhood of critical buses with protected PMUs, keeping the grid observable after removal and setting the wait window from the delay spread matter as much as the detection test.**

***Index Terms*—Co-simulation, cyber-physical systems, false data injection, GridPACK, HELICS, NS-3, phasor data concentrator, phasor measurement unit, state estimation, testbed.**

I. Introduction

FOR decades, transmission control centers have watched the grid through supervisory control and data acquisition (SCADA). Remote terminal units report unsynchronized magnitudes every few seconds, and the state estimator needs iterations over a nonlinear model to turn them into bus voltages [1]. A phasor measurement unit (PMU) changes this picture. It samples the voltage at its bus and the currents on the branches connected to it, stamps each frame with time from the global positioning system (GPS), and sends 30 to 60 frames per second to a phasor data concentrator (PDC) [2], [3]. The PDC aligns the frames by time stamp and passes sets to applications in the control center. Because the phasors are linear in the bus voltages, the control center can estimate the state in one step and act on it within a fraction of a second [1], [2].

Two things limit what the control center can do with these data. The first is the communication network. A frame that arrives after the PDC has released its time stamp is lost for that set, and a set that misses PMUs can leave part of the grid unobserved [4], [5]. The second is trust. An attacker who can change frames can try to hide a real problem or create a false one. Liu *et al.* [6] showed that an attacker who knows the measurement model can add an error that the residual test of the estimator cannot see, and later studies examined how many measurements such an attack needs, how to protect them and how to catch the attack with learned detectors [7]–[10]. Attacks on timing are just as effective. Shifting the time reference of a PMU or delaying its frames makes the PDC drop them [11], [12].

These questions cross the boundary between the power system and the network, so studying them requires both to be simulated together, with the PMUs, the PDC and the control center in between. Real-time hardware testbeds do this with high fidelity [13]–[16], but they cost a great deal, and the number of experiments they can run at once is fixed by the hardware. Open-source co-simulation with HELICS [17], [18] lowers the cost, and HELICS was designed with wide-area monitoring, protection and control among its use cases [17], so coupling a transmission solver with a network simulator is not new in itself. The recent published platforms, however, model distribution feeders, advanced metering or distributed energy resources [19]–[22], or couple transmission and distribution without any network [23]. Our own earlier prototype coupled GridPACK and NS-3 on the IEEE 14-bus system and studied distributed denial of service (DDoS), but it had no PMU chain, no estimator and no closed loop [24]. These platforms leave a practical problem open. A utility or university that plans a synchrophasor network must decide where to place PMUs, how long the PDC should wait for late frames, and what the control center should do when its bad data test raises an alarm. These choices interact with each other and with the type of disturbance. A short wait window may be harmless after a slow load change and decisive after a fault, and an attack that a well-placed PMU set exposes may pass unnoticed with a minimal one. Answering such questions needs repeatable experiments on a transmission system, with the grid, the network, the PMUs, the PDC and the control center all in the loop, run many times over different faults, attacks and random conditions. It also needs a facility that several students and

The authors are with the Department of Electrical and Computer Engineering, North South University, Dhaka 1229, Bangladesh (e-mail: mazharul.islam1@northsouth.edu; sunjare.zulfiker@northsouth.edu; masum.reza@northsouth.edu; hafiz.rahman@northsouth.edu).

Corresponding author: Hafiz Abdur Rahman (e-mail: hafiz.rahman@northsouth.edu).

This work was supported by the Bangladesh Energy and Power Research Council (EPRC) under grant EPRC/58-2018-007-01.

engineers can use at once without buying real-time simulators. None of the platforms above offers this combination, and none reports, for the same transmission system, how the PDC wait window, the spread of link delays, the redundancy of PMU placement and the treatment of detected bad data change the outcome of an attack across different fault types.

This paper presents such a testbed and uses it for a systematic study. GridPACK [25] solves the transmission grid as a message passing interface (MPI) job on a computing cluster, NS-3 [26] carries every PMU frame and every control command, and a control center estimates the state, tests and cleans it, and corrects voltage violations. HELICS [18] synchronizes the parts, and users operate the testbed from a Node-RED [27] dashboard in a web browser. Its contribution is the integrated workflow and the systematic assessment it enables. To the best of our knowledge, no published open-source testbed combines, at transmission level, explicitly modeled PMU and PDC timing, attacked measurements, a defended estimator and closed-loop voltage control, and lets remote users share a cluster through a queue that respects its computing resources. The work has priority for groups that are building PMU networks and research facilities with limited budgets, because it shows which design choices decide whether a fault is seen before any hardware is bought. Its scope is the IEEE 118-bus system with five fault types (an automatic voltage regulator (AVR) fault, a load step, a line trip, a generator trip and a three-phase bus fault), four attack types (simple and stealthy false data injection (FDI), delay and drop), two PMU placements, 20 further fault locations, and sweeps of link latency and PDC wait window, with several random seeds. The main contributions are as follows.

- *An open-source synchrophasor testbed.* The grid solver runs as a persistent parallel job, every PMU has its own network link to a PDC with a wait window, and the control center combines state estimation, a chi-square test with removal of the suspect PMU, and voltage control. We derive the closed-loop latency that the time coupling creates and confirm it in the logs.
- *Remote, multi-user operation on a cluster.* A queue starts an experiment only when its parallel ranks fit in the free slots, each experiment gets its own co-simulation broker, federates are placed on chosen nodes, and an observer streams every run to a web dashboard. Every run is kept as plain files.
- *A systematic study of faults and attacks.* The study covers 1087 runs, with every attack type tested against every fault type and at 22 fault locations under both placements, a matched-noise test of the detector, plus baselines, ablations and statistical tests that separate the effect of each defense.
- *Design findings with released evidence.* An attack that withholds or shifts the measurements of a faulted bus hides the fault unless the fault also shows at a bus that unattacked PMUs observe, and topological redundancy rarely changes this. Removing a detected PMU helps only if the bus stays observable. The PDC wait window must follow the spread of link delays, and voltage control needs a time filter so that it does not react to faults that clear on their own. The code, scenario files, scripts and run summaries are public so that every number can be regenerated.

The rest of this paper is organized as follows. Section II gives the background on synchrophasors, PMU state estimation, attacks and co-simulation. Section III reviews related testbeds and studies by theme and compares them in Table I. Section IV presents the proposed testbed and its algorithms. Section V describes the implementation, the experimental setup, the baselines and the metrics. Section VI reports the results, including ablations and statistical tests. Section VII discusses the findings and the threats to validity, and Section VIII concludes the paper.

II. Background

A. Synchrophasors, PMUs and PDCs

A synchrophasor is the phasor of a voltage or current referred to a common time reference. IEEE C37.118.1 defines its measurement and accuracy, and C37.118.2 the frames that carry it [3], [28]. A PMU reports a frame at a fixed rate $1/T$, for example $T = 1/30\,\mathrm{s}$. Frames from many PMUs reach a PDC, which buffers them by time stamp. Because a PDC cannot wait forever for a late frame, it releases a set when all expected PMUs have reported or when a wait time has passed [4]. The wait time trades latency for completeness, and in hierarchical networks the wait times of several PDCs have to be set together [5]. Measured end-to-end latencies of wide-area systems are in the tens of milliseconds [29], [30].

B. PMU State Estimation and Bad Data

If every PMU channel is a linear function of the complex bus voltages $V \in \mathbb{C}^n$, the m received channels satisfy $z = HV + e$, where e is the measurement error. The weighted least-squares (WLS) estimate minimizes the weighted squared residual and has a closed form [1]. Under Gaussian errors, the weighted sum of squared residuals J follows a chi-square distribution with $2m - 2n$ degrees of freedom when the data are good, so J above a quantile of that distribution signals bad data. The largest normalized residual then points to the measurement most likely to be wrong [1], [31]. A grid is observable from PMUs when every bus has a PMU or is adjacent to one, and the minimum set of PMUs that achieves this is found by integer programming [32], [33].

C. Attacks on Synchrophasor Data

An attacker who adds $a = Hc$ to the measurements, for any c, shifts the estimate by c and leaves the residual unchanged, so the chi-square test cannot see it [6]. Such a stealthy attack needs every measurement that depends on the shifted states. Kosut *et al.* [7] and Bobba *et al.* [8] studied how many measurements an attacker needs and which ones a defender should protect. Recent work builds AC attacks that pass the test with high probability [34] and detectors that learn the normal data to catch them [9], [10]. Attacks can also withhold data. Time-synchronization attacks and added delay

make frames miss the PDC [11], [12], [35], and flooding a PMU's connection can close it [36]. Placement can help. If the grid stays observable after the attacked PMUs are removed, the attack can be located and corrected [37], and placement can be chosen to make attacks harder [38].

D. Co-Simulation of Power and Communication Systems

A co-simulation runs separate simulators for the grid and the network and exchanges data and time between them. Early platforms such as EPOCHS [39] synchronized at fixed points, GECO [40] used a global event queue, and a survey [41] compares the coupling methods. Mosaik [42] and HELICS [17], [18] are general frameworks. In HELICS each simulator is a federate that publishes values and requests the next time it needs. A broker grants a time only when no other federate can still send an earlier message, which keeps causality across federates written in different languages [18]. Real-time co-simulation adds the constraint of wall-clock pacing and its own interfacing problems [43], [44].

III. Related Work

A. Hardware and Real-Time Testbeds

Hahn *et al.* [45] set out the architecture and evaluation of cyber-physical security testbeds, and PowerCyber [46] opened such a testbed with real-time simulators and field devices to remote users. NEFTSec [47] federated testbeds at several sites to study wide-area protection. Recent real-time platforms couple OPAL-RT or RTDS simulators with network emulators. Kumar *et al.* [13] protected wide-area damping control against FDI and lost signals with a learned signal reconstruction. Nguyen *et al.* [14] automated many attack scenarios on a large-scale testbed. Das *et al.* [15] rebuilt IEEE C37.118.2 frames to inject false data against bad data detection. Mishchenko *et al.* [16] demonstrated attacks on synchrophasor streams in a multidomain testbed and left the defender side to future work. Others built hybrid physical laboratories [48], training testbeds [49], hardware-in-the-loop platforms with commercial network simulators [50], and a cloud cyber range [51]. These systems give high fidelity, but each depends on licensed hardware at one site, and the number of experiments that can run at once is fixed by that hardware.

B. Open-Source Co-Simulation Platforms

HELICS has become the common choice for open-source co-simulation [18]. NATIG [19] packages GridLAB-D, NS-3 and HELICS in containers and generates attacks on inverter set-points in distribution microgrids. GridSecSim [20] couples GridLAB-D and NS-3 for advanced metering and trains an intrusion detector. Vallam Kondu *et al.* [21] attach an NS-3 module to a distributed energy resource management system and measure the communication impact of attacks. Masood *et al.* [22] show that a denial of service can mask the effect of a false data injection on a distribution grid. Zheng *et al.* [23] keep GridPACK's MPI parallelism inside a HELICS federation to co-simulate transmission and distribution faster than real time, but without a network. Our earlier prototype [24] added a Node-RED front end to GridPACK and NS-3 on the IEEE 14-bus system. None of these platforms models a PMU, a PDC and a transmission state estimator in the loop.

C. Synchrophasor Networks, Latency and the PDC

Several studies treat the network that carries synchrophasors. Castello *et al.* [29] built a validated platform for measuring the latency of a transmission operator's wide-area system. Morato *et al.* [30] simulated PMU traffic over 5G and found mean latencies near 20 ms. Pourramezan *et al.* [5] optimized the wait times of hierarchical PDCs for throughput and observability. Jha *et al.* [52] modelled the resiliency of synchrophasor networks as a graph and checked it in ns-3. Zadsar *et al.* [11] showed that the PDC wait time is itself a defense against time-synchronization attacks. These works study the network or the PDC alone and do not close the loop through an estimator and a controller.

D. Attacks and Defenses on PMU State Estimation

Detection studies usually work on recorded or generated data sets. Chen *et al.* [9] and Bhattacharjee *et al.* [10] detect stealthy FDI with unsupervised and self-supervised models on IEEE test systems including the 118-bus system. Yang *et al.* [34] built AC attacks that passed a chi-square test in 99.3% of cases. Khalafi *et al.* [37] placed PMUs so that the grid stays observable without any one PMU and located attacks by repeated estimation. Pourahmad *et al.* [38] placed PMUs and PDCs to reduce attack possibilities and communication cost on the IEEE 30- and 118-bus systems. Shereen and Dán [12] bounded the time references an undetectable time-synchronization attack must control. These methods are evaluated offline, without a simulated network or a controller that acts on the estimate.

E. Comparison and Research Gap

Table I summarizes representative works by method, test system, metrics, main result and limitation. Three gaps stand out. Testbeds that model the synchrophasor chain rely on real-time hardware at one site. Open-source platforms with a network simulator model distribution systems and have no PMU estimator in the loop. And none of the studies measures, on one transmission system, how the PDC wait window, the spread of link delays, the redundancy of placement and the treatment of detected bad data together decide whether an attack hides a fault from the controller. The testbed and the study in this paper address these three gaps.

IV. Proposed Testbed

A. Architecture

Fig. 2 shows the architecture of one experiment. Three federates form the closed loop. The grid federate drives GridPACK and publishes the true phasors that each PMU would measure. The network federate, an NS-3 program, turns them into PMU frames, carries them over a link per PMU to a PDC, and releases time-aligned sets. The control-center federate estimates the state, tests it, and sends generator

TABLE I
COMPARISON OF RELATED TESTBEDS AND STUDIES

Work	Method or platform	Test system	Metrics	Main result	Limitation
[13]	OPAL-RT hardware in the loop, CNN-LSTM reconstruction of PMU feedback	105-bus Indian grid	Damping of inter-area modes	Damping kept under FDI and lost signals	Licensed hardware, network not simulated
[14]	OPAL-RT and RTDS with EXata emulation, scripted scenarios	IEEE test feeders	Impact on operation	Many attack scenarios automated	Licensed tools, no synchrophasor estimation
[15]	Real-time simulator, C37.118.2 frame rewriting, virtual machines	IEEE 13-node feeder	Bad data detection	FDI on PMU data shown against detection	Hardware, no closed loop, PDC timing not studied
[16]	OPAL-RT with virtual PMUs and adversary setup	Aggregated dynamic model	Attack feasibility	Attacks on C37.118.2 streams demonstrated	Defender side left to future work
[19]	GridLAB-D, NS-3 and HELICS in containers	IEEE 123-bus, 9500-node	Voltage and frequency deviation	Inverter types react differently to set-point attacks	Distribution only, no PMUs or estimator
[20]	GridLAB-D, NS-3 and HELICS, intrusion detector on a single-board computer	IEEE 13-node, ten meters	Precision, recall, F1	F1 of 99.5% on metering traffic	Metering only, no grid control
[23]	GridPACK on MPI with GridLAB-D under HELICS	3000-bus with 100 feeders	Run time, accuracy	30 s simulated in 24.4 s on 4 processes	No network or attacks
[24]	GridPACK, NS-3, HELICS, Node-RED	IEEE 14-bus	Latency, loss, response time	Latency rose from 10 to about 100 ms under DDoS	DDoS only, no PMU chain or control
[5]	Model of hierarchical PDC wait-time policies	Realistic PMU data	Throughput, observability	Optimal wait times found	No attacks, no closed loop
[11]	Co-optimized PMU–PDC tree and PDC wait time	Kundur, IEEE 39-bus	Modal observability	Wait time limits data dropout under attack	Model-based, network not simulated
[37]	N-1 placement, repeated estimation, clustering	Sample system	Detection, localization	Stealth impossible if observable after removal	Static, no network
[9]	Unsupervised detector of stealthy FDI	IEEE 39- and 118-bus	Detection performance	Catches attacks missed by residual tests	Offline data, no network
[34]	AC FDI by tensor principal component analysis	IEEE 5- to 3120-bus	Attack success rate	99.3% success against chi-square test	Attack side only
This work	GridPACK on MPI, NS-3 PMUs and PDC, WLS with chi-square test and removal, voltage control, HELICS, remote dashboard	IEEE 118-bus, five fault types, 22 locations	Violation time, complete and observable sets, alarms, latency, run time	Attacks hide faults unless an observed neighbour exposes them; removal must keep observability; PDC wait must cover delay spread	Not paced to wall-clock time

setpoint commands, which return through NS-3 to the grid. A fourth federate, the observer, only subscribes and copies the state of the run to the dashboard, so no federate in the loop waits for it. Table II lists the HELICS publications. Around the federation, a FastAPI service [53] queues experiments, starts the broker and the federates, and serves results to the Node-RED dashboard used by remote users. Fig. 9 shows how these parts are placed on the cluster used in this paper. Fig. 1 follows one pass around the closed loop. Measurements travel from the grid through the PMUs, their links and the PDC to the control center, which estimates and tests the state and sends a setpoint command back through the network to the generator. The attacks act on the PMUs and on their links.

TABLE II
HELICS PUBLICATIONS

Topic	From, to	Content
grid/meas	grid, NS-3	True phasors of every PMU channel, each grid step
ns3/pdc	NS-3, control center	Time-aligned PMU sets released by the PDC
cc/commands	control center, NS-3	Generator setpoint commands
ns3/cmd_delivered	NS-3, grid	Commands that reached the generator node, with arrival time
grid/status	grid, observer	True bus voltages, commands applied, solver report
ns3/status	NS-3, observer	Per-PMU frame counters, PDC and command counters
cc/status	control center, observer	Test result, removed PMUs, estimate, computing time

B. Grid Federate and the GridPACK Server

GridPACK distributes a network over MPI processes, partitions it with ParMETIS and solves the network equations with PETSc [25], [54]. Its applications normally run once per study. A co-simulation needs a new solution at every grid step, and restarting an MPI job each step would spend most of the time on start-up and partitioning. We therefore wrote a small server, `pf_server`, around GridPACK's power flow. It is started once per experiment with `mpirun`, reads the case and partitions the network once, and then waits for commands on the standard input of rank 0 (Table III). Rank 0 broadcasts each batch, each rank applies the changes to the buses it owns, GridPACK solves the power flow by Newton–Raphson with the MUMPS direct solver [55], and the bus voltages are gathered to rank 0 [56]. Replies start with `@@` so that the driver can separate them from the solver's own output, which it passes to the dashboard. Zheng *et al.* [23]

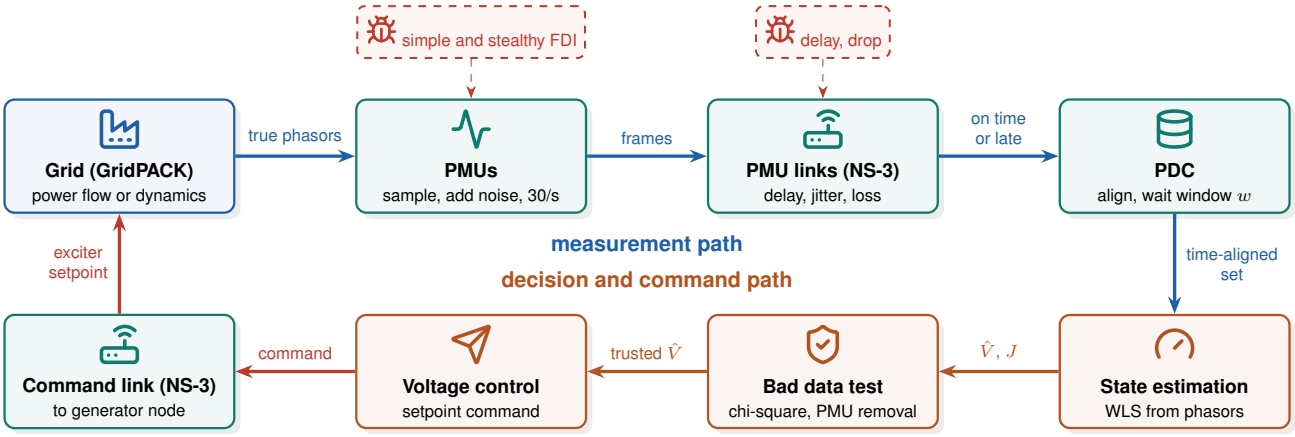


Fig. 1. Closed monitoring and control loop and the points where the attacks act.

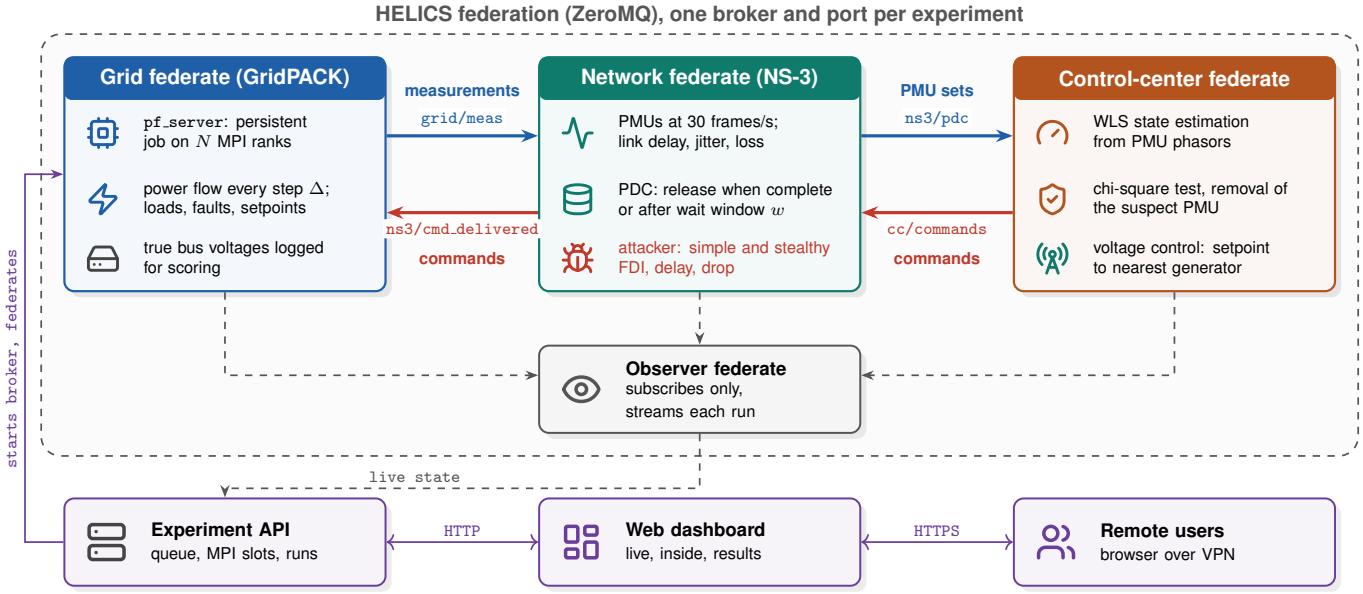


Fig. 2. Testbed architecture for one experiment.

TABLE III
PROTOCOL OF PF_SERVER, ONE EXCHANGE PER GRID STEP

Direction	Line	Meaning
to rank 0	LOAD bus P Q	new load at a bus
to rank 0	VSET bus id V	new generator setpoint
to rank 0	SOLVE / QUIT	solve the batch / stop
from rank 0	@@RANK rank host buses branches	partition of each rank
from rank 0	@@READY nbus nranks t_setup	network read, partitioned
from rank 0	@@RESULT ok nbus t_apply t_solve	solve finished, timings
from rank 0	@@V bus Vm Va … @@END	voltage of every bus

kept GridPACK's parallelism under HELICS in a similar way by making rank 0 the federate. Our server keeps HELICS out of the MPI program, so the same driver can fall back to a built-in solver if the MPI job fails.

The grid federate is a Python program that drives the server. At every grid step Δ it applies the commands that have arrived over the network, updates the loads, solves, publishes the phasors, and logs the true voltage of every bus so that the experiment can be scored afterwards. Each load follows a slow sinusoid plus an independent random walk, which keeps the measurements from being constant. Events are applied at their scheduled grid step. In this quasi-steady-state (QSS) mode, with $\Delta = 0.5$ s, the testbed offers an AVR fault, which moves a generator's voltage setpoint to a wrong value, and a load step.

Faults that need machine dynamics run in a second, dynamic mode. A companion server, dsf_server, keeps GridPACK's dynamic simulation alive on the same MPI ranks and advances it with an integration step of 5 ms. The grid federate then steps once per PMU frame, so $\Delta = T$. Generators without dynamic data in the case file get typical models: a round-rotor machine (GENROU), a simple exciter (SEXS) and a steam governor (TGOV1). The dynamic mode offers a line trip, a

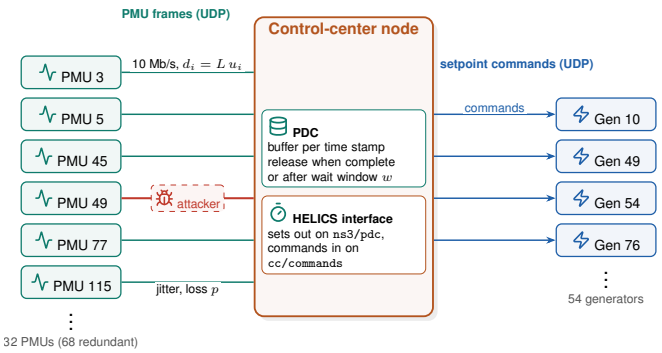


Fig. 3. NS-3 model of the synchrophasor network.

generator trip and a three-phase bus fault cleared after a given time. Loads are constant impedances, and voltage setpoint commands act on the exciter reference.

C. Network Federate: PMUs, Links and PDC

Every PMU is an NS-3 node with its own point-to-point link to the control-center node. The one-way delay of link i is drawn once per experiment as

$$d_i = L\,u_i, \qquad u_i \sim \mathcal{U}(0.5,\,1.5), \tag{1}$$

where L is the mean latency. Each frame also waits 2 ms of processing plus a jitter drawn from $\mathcal{U}(0,2)$ ms, and a packet error model loses frames with a fixed probability. Each generator has its own node and link, which carry commands. Fig. 3 shows this model for the minimum placement.

At every frame time kT, each PMU takes the latest phasors published by the grid federate and adds noise. With s the true phasor and $\epsilon_1, \epsilon_2 \sim \mathcal{N}(0,1)$, the reported phasor is

$$p = |s|\,(1 + 0.001\,\epsilon_1)\,e^{\,j(\angle s + 0.001\,\epsilon_2)}. \tag{2}$$

The PMU sends the frame as a user datagram protocol (UDP) packet carrying its number, time stamp and phasors. These are abstract PMU frames: they carry the measurement content but not the IEEE C37.118.2 encoding [28]. The PDC keeps a buffer per time stamp and releases a set as soon as every PMU has reported, or a wait window w after the first frame of that time stamp arrived [4]. A frame that arrives after its set was released is counted as late. Algorithm 1 gives one frame interval, including the attacks of Section IV-F.

D. PMU Placement

A PMU at bus i measures the voltage of i and the current of every in-service branch at i, so it makes i and its neighbours observable. With A the bus connectivity matrix including the diagonal and $x \in \{0,1\}^n$ the placement, the testbed solves

$$\min_{x}\ \mathbf{1}^{\top}x \quad \text{s.t.} \quad Ax \geq \kappa, \tag{3}$$

as in [33], where $\kappa_i = \min(k, \deg(i)+1)$. Here $k=1$ gives the minimum placement for full observability and $k=2$ a redundant placement in which every bus is seen twice where its degree allows. The problem is solved with the HiGHS solver

Algorithm 1 Network federate, frame interval k

Input: frame period T, link delays d_i, wait window w, attack settings

request HELICS time kT
publish the PDC sets and command deliveries of the last interval
read new grid phasors s_i and commands; send each command to its generator node
$a \leftarrow$ attack active at kT
if a is an FDI and the attacker holds the PMU at target bus b **then**
  $c \leftarrow (v_f - |s_{b,0}|)\,e^{j\angle s_{b,0}}$ ▷ adaptive offset
end if
for each PMU i **do**
  $p \leftarrow s_i$ with noise by (2); $e \leftarrow 0$
  if a and PMU i is attacked **then**
    if drop **then** mark the frame dropped; **continue**
    end if
    if FDI **then** add $h_{kb}\,c$ to attacked channels
    end if
    if delay **then** $e \leftarrow d_a$
    end if
  end if
  send p after $2\,\text{ms} + \mathcal{U}(0,2)\,\text{ms} + e$; the link adds d_i
end for
run NS-3 events until $(k+1)T$; for each frame of time stamp k' that reaches the PDC: if set k' was released, mark it late; else store it, release set k' if complete, and if it is the first frame of k', schedule the release at arrival $+\,w$

[57]. For the IEEE 118-bus system the minimum placement has 32 PMUs with 169 phasor channels (Fig. 4) and the redundant placement 68 PMUs with 315 channels (Fig. 5). Bus 49, where the fault and the attacks of Section V take place, has a PMU in both. Under the minimum placement it is observed by PMUs 45 and 49, and under the redundant placement by PMUs 42, 45, 49, 51 and 66. This redundancy is topological. It counts the PMUs that observe each bus, but it does not by itself set how many PMUs an attacker must hold to hide a change at a bus, which depends on the neighbourhood of that bus (Section VI-D).

E. Control Center

State estimation. With all quantities complex, the channels of the received PMUs satisfy $z = HV + e$, where a row of H is a unit vector for a voltage channel and a row of the branch admittance matrices for a current channel. The WLS estimate is

$$\hat{V} = G^{-1}H^{\mathsf{H}}Wz, \qquad G = H^{\mathsf{H}}WH, \tag{4}$$

with $W = \text{diag}(1/\sigma_k^2)$ and $\sigma_k = \max(\sigma|z_k|, 10^{-4})$. G is Hermitian positive definite and is factorized once per set with a sparse LU in symmetric mode. A set is observable when every bus is the own bus or a neighbour of a received PMU. When it is not, the estimator adds weak pseudo-measurements

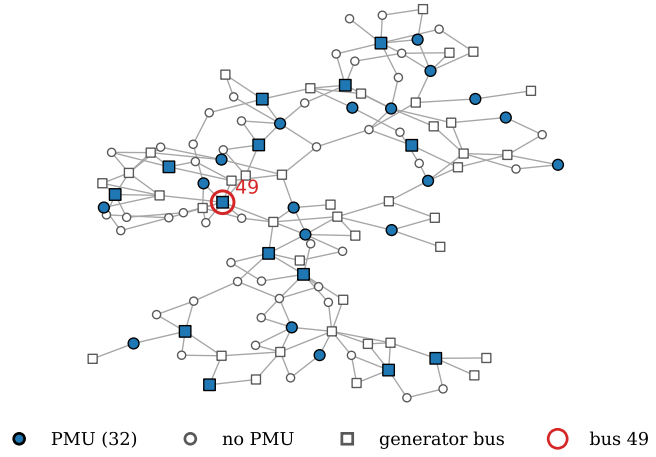


Fig. 4. Minimum placement, 32 PMUs.

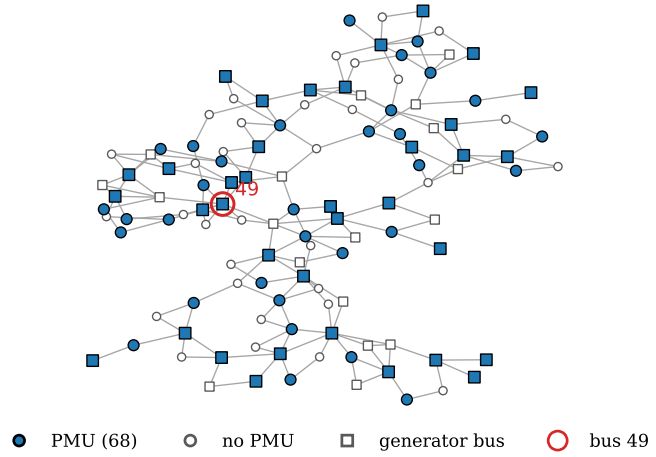


Fig. 5. Redundant placement, 68 PMUs.

equal to the previous estimate with standard deviation 0.05 pu, so that it still returns a full state, and the set is not tested.

Bad data detection and removal. For an observable set, the statistic

$$J = \sum_{k=1}^{m} \frac{|z_k - h_k \hat{V}|^2}{\sigma_k^2} \tag{5}$$

is compared with $\tau = \chi^2_{2m-2n}(1-\alpha)$ [1]. For the IEEE 118-bus system with $\alpha = 0.01$, $\tau = 138.1$ with minimum placement and 462.2 with redundant placement. On an alarm, the control center computes the normalized residuals [31]

$$r_k^N = \frac{|r_k|}{\sqrt{\Omega_{kk}}}, \qquad \Omega = R - HG^{-1}H^{\mathsf{H}}, \tag{6}$$

with $R = W^{-1}$, removes all channels of the PMU that holds the largest one, estimates again and repeats the test, up to three PMUs. Removing every channel of that PMU matches the threat, since an attacker who controls a PMU controls all its channels.

Voltage control. If the final estimate is free of alarms, the control center checks every bus against the limits $[v_{\min}, v_{\max}]$ with a dead band δ. For a bus outside the limits it picks the in-service generator with the fewest branches to that bus and moves its setpoint by one step in the direction that corrects the violation, within the setpoint range and at most once per cooldown period per generator. The command carries an absolute setpoint. The step is taken from the control center's own record of the setpoint, which can be set in two ways. In the *dispatch* mode, used unless stated otherwise, the record is the dispatched value from the case file, as in a control center that does not see the reference inside the exciter. In the *telemetry* mode, the record follows the reference actually set at the generator, as a SCADA reading of the regulator would report it. The two modes differ only after an AVR fault, which changes the reference without the control center's knowledge. While an alarm remains after removal, no command is sent, because the estimate cannot be trusted. Algorithm 2 gives the procedure for one set.

Algorithm 2 Control center, one PDC set

Input: set S of received PMUs, H, σ, α, limits $[v_{\min}, v_{\max}]$, dead band δ, step Δv

1: $R \leftarrow \emptyset$ ▷ removed PMUs
2: **if** S is observable **then** estimate $\hat{V}$ by (4); compute J by (5) and τ; $alarm \leftarrow J > \tau$
3: **else** add pseudo-measurements, estimate $\hat{V}$; $alarm \leftarrow$ false ▷ not testable
4: **while** $alarm$ and $|R| < 3$ and removal is enabled **do**
5: compute r^N by (6); $p \leftarrow$ PMU with the largest r_k^N
6: $R \leftarrow R \cup \{p\}$; drop all channels of p from S
7: re-estimate and re-test as in lines 2–3
8: **end while**
9: **if** $alarm$ **then**
10: **return** ▷ hold control while bad data remain
11: **end if**
12: **for** each bus i with $|\hat{V}_i| > v_{\max} + \delta$ or $|\hat{V}_i| < v_{\min} - \delta$ **do**
13: $g \leftarrow$ in-service generator nearest to i in branches
14: **if** no command was sent to g within the cooldown **then**
15: $v_g \leftarrow v_g \mp \Delta v$, clipped to the setpoint range
16: publish (g, v_g) on `cc/commands`
17: **end if**
18: **end for**

F. Attack Models

The attacker targets a bus b and wants the control center to see $|V_b| = v_f$. In the linear model this is the state change $\Delta V = c\, e_b$ with $c = (v_f - |V_b|)\, e^{j\theta_b}$, which changes channel k by $h_{kb}\, c$. Four attacks act on the frames of the attacked PMUs between a start and an end time. The *simple FDI* changes one channel, the voltage channel of the PMU at b if there is one. The other channels still describe the true state, so the set becomes inconsistent. The *stealthy FDI* adds $a = H\Delta V$, that is $h_{kb}\, c$ to every channel with $h_{kb} \neq 0$. Then the residual is unchanged and so is J [6]. It needs every PMU that has such a channel. In both cases, an attacker who holds the PMU at b reads the true V_b from each frame and recomputes c, so the injection follows the grid. The *delay* attack holds the frames

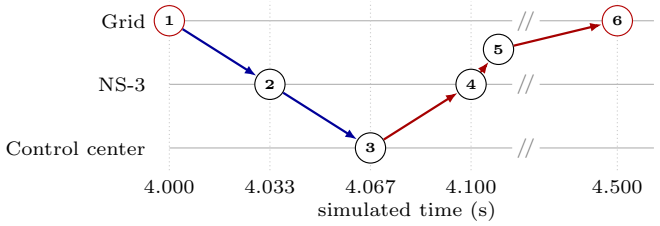


Fig. 6. Logged closed-loop timeline after the AVR fault at 4 s.

of every PMU with a channel that depends on V_b for an extra d_a, and the *drop* attack discards them. The attacker knows H and the operating point, which is the full-knowledge case of [6] and a bound on what the defenses can do. That the stealthy injection passes a residual test is known [6]. What the testbed adds is its consequence in the closed loop, with the network and the PDC in between.

G. Time Coupling and Closed-Loop Latency

All federates use the HELICS ZeroMQ core. The grid federate requests the times $j\Delta$. The network federate requests the frame times kT one after another, runs the NS-3 event queue up to the next frame time, and at each granted time first publishes what the previous interval produced and then reads new phasors and commands. The control center does not step on its own. HELICS wakes it whenever a new PDC set is published. The grid and network federates are uninterruptible, so they are granted only the times they request.

This scheme fixes the latency of the loop. Suppose an event changes the grid at $t_0 = j\Delta$. Both the grid and the network federate are granted t_0, and the network federate samples the phasors that were current before the grid published its new solution, so the event first appears in the frame stamped t_0+T. If the PDC releases that set r after its time stamp, the network federate publishes it at the next frame time, $t_0+T+T\lceil r/T\rceil$, and the control center issues its command at that time. NS-3 reads the command at its next step and delivers it after the generator link delay d_g, and the grid applies it at its next step. The first command therefore takes effect at

$$t_{\text{apply}} = \Delta \left\lceil \frac{t_0 + T\,(2 + \lceil r/T \rceil) + d_g + \epsilon}{\Delta} \right\rceil , \qquad (7)$$

where ϵ is the processing and jitter delay. The release delay r grows with the wait window and the link delays, as Section VI shows. With $T = 33.3\,\text{ms}$, $\Delta = 0.5\,\text{s}$, $r \le T$ and d_g of a few tens of milliseconds, the first command takes effect one grid step after the event. Whether one command clears the violation depends on the size of the setpoint step needed. Fig. 6 shows the timeline logged in the base experiment. The one-frame lag between grid and PMUs is a property of the time coupling. It could be removed by letting the network federate request a time just after the grid's, at the cost of a second synchronization per frame. We kept the simpler scheme and report the lag.

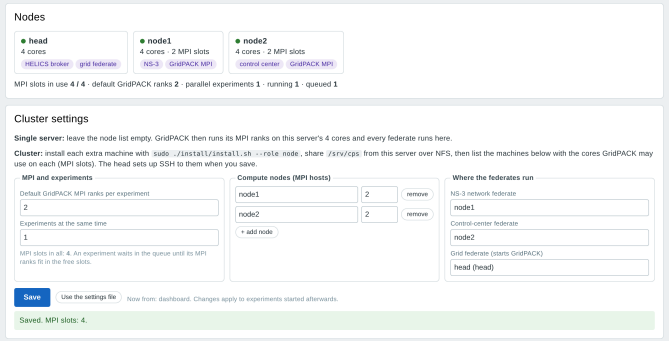


Fig. 7. Cluster page of the dashboard.

H. Operating Layer and Dashboard

An experiment is described by a scenario: grid, duration, MPI ranks, PMU placement and rate, network parameters, event, attack and control settings. Scenarios are entered in a form on the dashboard or uploaded as YAML files of up to 50 experiments that share common fields. The service checks each scenario against the chosen grid and rejects it with the name of the wrong field.

The service runs a set number of experiments at the same time, but starts an experiment only when its GridPACK ranks fit in the free MPI slots of the compute nodes. MPI ranks busy-wait for messages, so oversubscribed cores would slow every run that shares them. For each experiment the service creates a run directory on the shared file system, picks a free HELICS port, writes the configuration of each federate and starts the broker and the federates over the secure shell (SSH) on the nodes set on the Cluster page (Fig. 7). GridPACK's ranks are spread over the nodes through an MPI host file. If a federate fails, the others are stopped and the end of its log is shown in the queue.

The observer writes the state of the run about twice per second. The Live page shows the true and estimated voltages, the chi-square test, PDC completeness and each command. The Inside page shows what each engine computes, such as the partition of the MPI ranks, the Newton–Raphson mismatch, frames per PMU and the test before and after removal (Fig. 8). The Results page opens finished runs. Every run is a directory of plain files (frames, sets, estimates, commands, true voltages and summaries), so results can be analysed outside the dashboard. All figures and tables of Section VI were produced from these files by scripts that are published with the paper. The dashboard and the application programming interface (API) are protected by authentication, and remote users reach the head node through the institution's virtual private network (VPN) or an SSH tunnel.

V. Implementation and Experimental Setup

A. Test System and Preprocessing

All experiments use the IEEE 118-bus system, taken from the MATPOWER case file [58]. It has 118 buses, 186 branches and 54 generators. The testbed converts the case to GridPACK's input format and to a topology file used by the placement, the estimator and the network federate, and checks

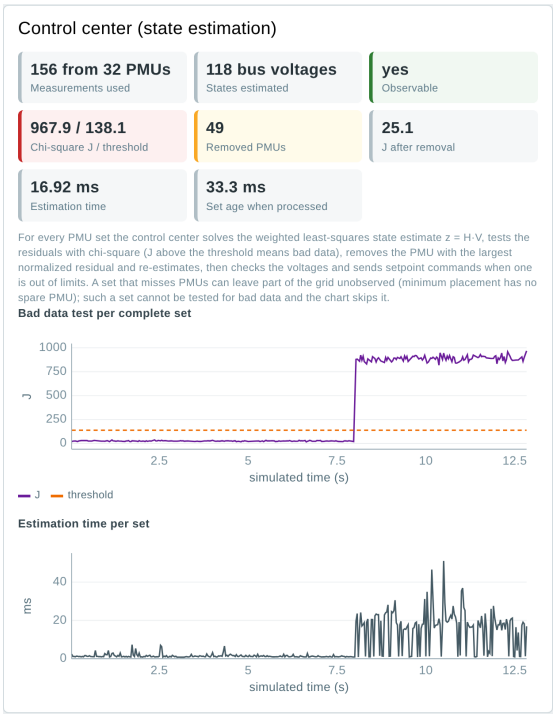


Fig. 8. Control-center panel of the Inside page during a simple FDI on bus 49.

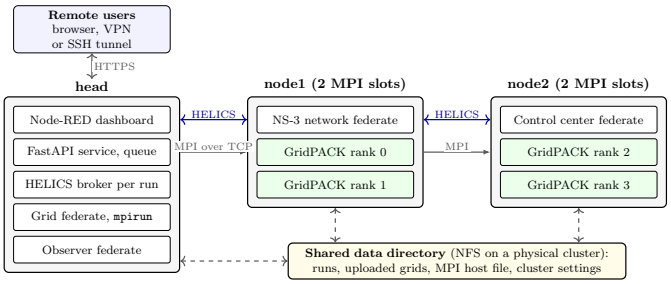


Fig. 9. Deployment on the three-node cluster.

that both solvers agree on the base case before a grid is accepted. No other data set is involved. Every measurement in the study is generated by the co-simulation.

B. Hardware and Software

The experiments ran on a three-node cluster: a head node and two compute nodes with two MPI slots each, connected by a network running the transmission control protocol and internet protocol (TCP/IP) and sharing the data directory. Every node runs the same installation, and the head node starts the federates over SSH and the GridPACK ranks through an MPI host file, as on any Linux cluster. For this study the three nodes were hosted on one server with a four-core Intel Xeon processor at 2.1 GHz and 15 GB of memory. Fig. 9 shows the placement of each part, and Table IV lists the software. Two experiments ran at a time. The same build scripts install a node natively on Ubuntu or as a container image.

TABLE IV
SOFTWARE USED IN THE TESTBED

Component	Software
Grid simulation	GridPACK 3.5, PETSc 3.19, MUMPS, ParMETIS
Parallel runtime	Open MPI 4.1.6
Network simulation	NS-3.48
Co-simulation	HELICS 3.6.1, ZeroMQ core
Control center, API	Python 3.12, NumPy, SciPy, FastAPI
Dashboard	Node-RED 4.1, Dashboard 2.0
Operating system	Ubuntu 24.04

C. Scenarios and Parameters

The base scenario lasts 10 s with $\Delta = 0.5$ s, 30 frames per second, $L = 10$ ms, $w = 20$ ms, no random loss, and GridPACK on one MPI rank. At 4 s the AVR of generator 49 fails and moves its setpoint from 1.025 to 1.12 pu, which raises bus 49 above the upper limit of 1.08 pu. Every attack targets bus 49 from 4 to 9 s. Both injections try to make the control center see $|V_{49}| = 1.02$ pu, a normal value. Table V lists the parameters. From this base, 21 scenarios vary one factor at a time: the attack (none, simple FDI, stealthy FDI, delay of 100 ms, drop), the placement (32 or 68 PMUs), random frame loss (0 or 5%), the mean latency L (5, 10, 20, 40, 80 ms) and, at $L = 40$ ms, the wait window w (10, 20, 40, 60, 80 ms). Each scenario ran with seeds 1 to 10. The seed sets the link delays (1), the jitter, the loss, the measurement noise (2) and the random walk of the loads. Another 15 runs repeated the base scenario with 1, 2 and 4 MPI ranks and seeds 1 to 5. All 225 runs were defined in one scenario file and submitted through the API.

Fault campaign. A second scenario file applies every attack to the other four fault types of the testbed, so that each attack meets each fault. The faults occur at 4 s: a load step of +200% at bus 53 in QSS mode, and, in dynamic mode, a trip of line 76–77, a trip of the generator at bus 80 and a three-phase fault at bus 49 cleared after 0.1 s. Without control each fault drives some bus voltage outside the limits. The attacks act from 4 to 9 s on the bus where the violation appears, bus 53, 76, 76 and 49 respectively, and the injections report $v_f = 0.97$ pu (1.0 pu for the bus fault). For each fault, eleven scenarios cover the two baselines, the simple FDI with and without removal, the stealthy FDI, delay and drop with 32 PMUs, and the four attacks with 68 PMUs, each with seeds 1 to 5, which adds 220 runs.

Location campaign. A third scenario file repeats the attacks at 20 more fault locations: AVR faults at 16 generator buses with degree 1 to 8, and load steps of +200% at load buses 21, 52, 95 and 118. Each location runs the two baselines and the four attacks under both placements with seed 1, ten runs per location, and each AVR location also runs the telemetry mode of the controller. With one more telemetry-mode run at generator 49, this gives $16 \times 11 + 4 \times 10 + 1 = 217$ runs.

Detection and seed campaigns. The main campaigns use an estimator standard deviation $\sigma = 0.002$, twice the simulated noise. To measure the detector itself, a fourth scenario file runs the grid without a fault and applies a simple FDI to the voltage

TABLE V
PARAMETERS OF THE BASE SCENARIO

Parameter	Value
Duration, grid step Δ	10 s, 0.5 s
PMU rate	30 frames/s
Mean link latency L, spread	10 ms, $\pm 50\%$
Processing delay, jitter	2 ms, $\mathcal{U}(0, 2)$ ms
PDC wait window w	20 ms
Noise	0.1% magnitude, 0.001 rad
Estimator σ, test level α	0.002, 0.01
Pseudo-measurement deviation	0.05 pu
Limits $[v_{\min}, v_{\max}]$, dead band δ	0.94–1.08 pu, 0.003 pu
Setpoint step, range, cooldown	0.01 pu, 0.95–1.10 pu, 1 s
Load variation	2% sinusoid, 20 s period, random walk
Event	AVR fault, generator 49, 1.12 pu at 4 s
Attack window, target, v_f	4–9 s, bus 49, 1.02 pu
Delay attack d_a	100 ms
GridPACK MPI ranks	1 (2 and 4 in Section VI-K)

channel of PMU 49 from 2 to 9 s with offsets of 0.002, 0.005, 0.01, 0.02 and 0.05 pu, plus attack-free runs, with $\sigma = 0.002$ and with the matched $\sigma = 0.001$, under both placements and with seeds 1 to 5 (120 runs). The same file repeats the AVR-fault attacks at bus 49 with the matched σ (25 runs). A fifth file repeats ten locations of the location campaign with seeds 2 to 5 for the attack-free control and the simple FDI, stealthy FDI and delay under both placements (280 runs). In all, the study comprises 1087 runs. In dynamic mode the violation time is measured at every frame, so its resolution is 33.3 ms.

D. Baselines and Ablations

Two baselines bound the results. *No control* runs the same fault with the estimator and test but without voltage control, and gives the violation that the loop must remove. *Control without attack* gives the best the loop can do. The proposed configuration, with the chi-square test, removal of up to three PMUs and voltage control, is then compared with three ablations. *Detection without removal* keeps the test but holds control on every alarm, as a control center without identification would. *Minimum versus redundant placement* removes the extra PMUs. *PDC wait window* and *link latency* sweeps change the timing of the PDC and the network. For the computing layer, the MPI sweep compares one, two and four ranks.

E. Metrics and Statistical Procedure

For each run we record the *violation time*, the time during which the true voltage of some bus was outside the limits. It is measured at the grid steps and so has a resolution of 0.5 s. We also record the time the first command was issued, the share of PDC sets that were complete and the share that were observable, and the number of sets whose first test raised an alarm and the number still in alarm after removal. On the network side we record the share of frames that arrived in time, their latency, and the mean delay of the PDC release after the time stamp. Finally we record the mean absolute error of the estimate and the GridPACK solve time per step. For each scenario we report the mean and standard deviation over the ten seeds and a 95% confidence interval from Student's t distribution. Differences between a scenario and its reference are tested with a two-sided exact permutation test on the difference of means, which needs no assumption about the distribution and remains valid when many runs give the same value. We also give the difference of means with a 95% confidence interval. When all runs of both groups give the same value, the test cannot reject equality, and we say so.

VI. RESULTS

A. Validation of the Grid Solution

The base scenario was run once more with the testbed's built-in Newton–Raphson solver in place of GridPACK. At all 21 grid steps, the true voltages of the 118 buses were identical to the six decimals logged, both for GridPACK on one rank and for GridPACK on four ranks split over the two compute nodes. The partition, the MPI exchange and the restart-free server therefore do not change the solution.

B. Closed Loop Without Attack

Fig. 10 shows bus 49 in the two baselines. Without control the fault keeps $|V_{49}|$ near 1.12 pu to the end of the run, a violation of 6.0 s. With control, the command for generator 49 was issued at 4.067 s, delivered at 4.107 s and applied at 4.5 s, as predicted by (7) and shown in Fig. 6. It set the reference of generator 49 to 1.015 pu, the dispatched 1.025 pu minus one step, and so replaced the faulty 1.12 pu in one command. The violation lasted one grid step, 0.5 s. In the telemetry mode the control center stepped from the faulty reference instead. Its first command was clipped to the upper setpoint limit of 1.10 pu, and two more commands at one-second intervals brought the reference to 1.08 pu at 6.5 s, so the violation lasted 2.5 s. The fast recovery in the dispatch mode therefore comes from overwriting the faulty reference, and a control center that only steps from what the regulator reports needs several cooldown periods. The same times were logged in all ten seeds. All 9568 frames of each run arrived in time, with a median latency of 12.8 ms and a 95th percentile of 17.3 ms (means over seeds), and the PDC released each set 17.9 ms after its time stamp on average. The mean absolute error of the estimate was 2.8×10^{-4} pu, and no set raised an alarm.

C. Attacks and PMU Placement

Table VI lists the 13 scenarios on attacks and placement, and Fig. 11 shows the violation time with its 95% confidence interval. In these scenarios the random factors changed the frames, delays and noise but never the outcome, so every seed gave the same violation time and the same command time.

Simple FDI with 32 PMUs. The attacker holds PMU 49 and replaces its voltage channel. In the set stamped 4.000 s the fault is not yet visible and the falsified 1.02 pu is close to the true 1.025 pu, so $J = 36$ (seed 1). In the next set the true voltage is 1.12 pu, the channel still reads 1.02 pu, and the current channels disagree with it. J rose to 2414

TABLE VI
ATTACKS AND PMU PLACEMENT, IEEE 118-BUS SYSTEM, AVR FAULT AT GENERATOR 49 AT 4 S, ATTACKS ON BUS 49 FROM 4 TO 9 S. MEAN ± STANDARD DEVIATION OVER TEN SEEDS; A SINGLE VALUE MEANS ALL SEEDS GAVE THE SAME RESULT

Case	Attacked PMUs (bus)	Complete sets (%)	Observable sets (%)	Alarm sets (first test)	Alarm sets (after removal)	First command issued (s)	Violation time (s)
Minimum placement, 32 PMUs							
No control (baseline)	–	100.0	100.0	0	0	–	6.0
Control, no attack (baseline)	–	100.0	100.0	0	0	4.067	0.5
Simple FDI	49	100.0	95.0	15	0	4.067	0.5
Simple FDI, no removal	49	100.0	100.0	149	149	9.033	5.5
Stealthy FDI	45, 49	100.0	100.0	0	0	4.067	0.5
Delay 100 ms	45, 49	49.7	49.7	0	0	9.033	5.5
Drop	45, 49	49.7	49.7	0	0	9.033	5.5
5% frame loss	–	20.5 ± 1.5	20.5 ± 1.5	0	0	4.067	0.5
Redundant placement, 68 PMUs							
Simple FDI	49	100.0	100.0	15	0	4.067	0.5
Stealthy FDI	42, 45, 49, 51, 66	100.0	100.0	0	0	4.067	0.5
Delay 100 ms	42, 45, 49, 51, 66	49.7	49.7	0	0	4.067	0.5
Drop	42, 45, 49, 51, 66	49.7	49.7	0	0	4.067	0.5
5% frame loss	–	3.0 ± 0.8	85.3 ± 2.0	0	0	4.067	0.5

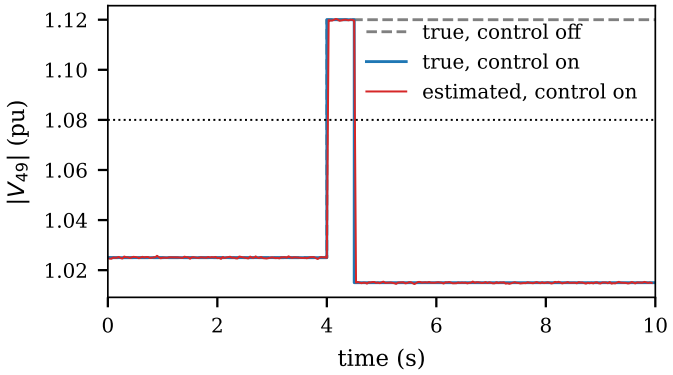


Fig. 10. Voltage of bus 49 with and without control (seed 1).

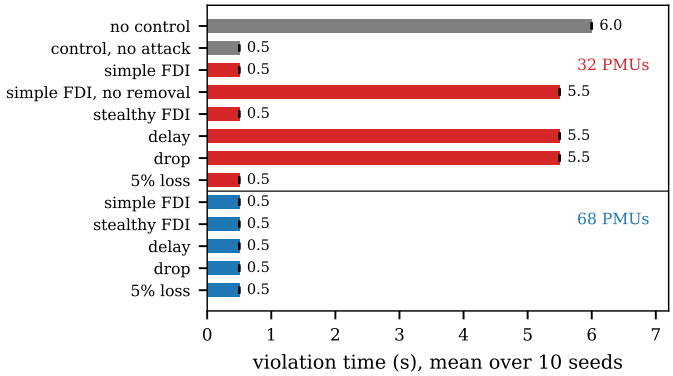


Fig. 11. Violation time per scenario, AVR fault (mean and 95% CI over ten seeds).

against a threshold of 138.1 (Fig. 13). The largest normalized residual pointed to PMU 49, which was removed, and the new estimate put bus 49 at 1.119 pu (Fig. 14). The command was issued at 4.067 s, as without attack. After the command took effect, the true voltage was 1.015 pu, close to the value the attacker reported, so J fell below the threshold for the rest of the attack. This is why exactly 15 sets, 0.5 s × 30 sets/s, raised an alarm in every seed. Removing PMU 49 left some buses without an observing PMU in those 15 sets, so the final estimate could not be tested again. The control center acted on an estimate completed with pseudo-measurements, which explains the 95.0% of observable sets. The command was correct, but the test that would have confirmed it was not run.

Detection without removal. With the same attack and the test in place but no removal, every set from 4.033 to 9.0 s raised an alarm. The estimate of bus 49 stayed near the truth, at 1.116 pu on average in seed 1, because the current channels outweighed the single false voltage channel. The control center nevertheless held its commands while the alarm stood. The command came only after the attack ended, at 9.033 s, and the violation lasted 5.5 s in every seed. Detection without identification protected the estimate and blocked the control.

Stealthy FDI. Bus 49 is observed by PMU 49 and by the current that PMU 45 measures on branch 45–49, so the stealthy attack changed both. As expected, J did not move and stayed below 36 during the attack (Fig. 13), and the estimate of bus 49 stayed at 1.02 pu (Fig. 12). The attack still failed to hide the fault. Shifting the estimate of bus 49 cannot also hide the fault's effect on the neighbouring buses, whose measurements are untouched. The control center estimated bus 48 at 1.096 pu, issued the command because of that bus at 4.067 s, and the command went to generator 49, the generator nearest to bus 48.

Delay and drop. Delaying the frames of PMUs 45 and 49 by 100 ms, five times the wait window, made them late for every set. Their sets were released without them, so 50.3% of the sets were incomplete and unobservable. Bus 49 was then estimated from pseudo-measurements of the last value seen before the attack, 1.025 pu (Fig. 12), and the violation went unseen until the frames returned at 9 s, giving 5.5 s of violation in every seed. Dropping the frames had the same effect. For the control center, a frame that arrives after the window is a lost frame.

Redundant placement. With 68 PMUs the simple FDI was detected in the same set and PMU 49 was removed. This time

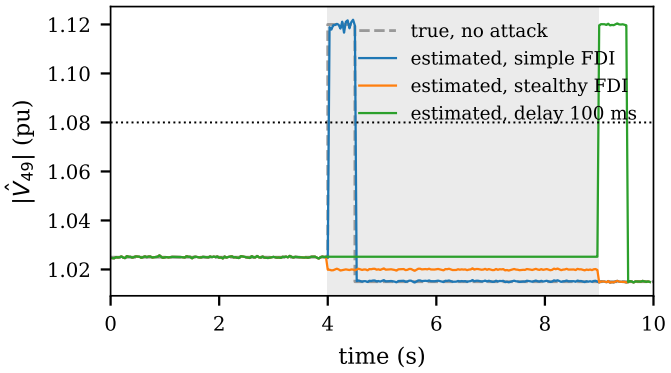


Fig. 12. Estimate of bus 49 under three attacks, 32 PMUs (seed 1).

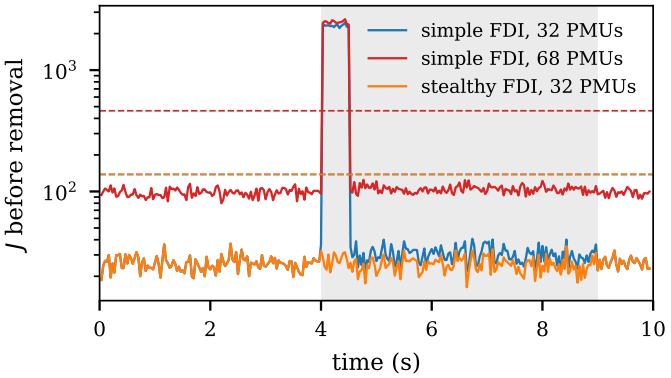


Fig. 13. Chi-square statistic J (solid) and threshold (dashed), seed 1.

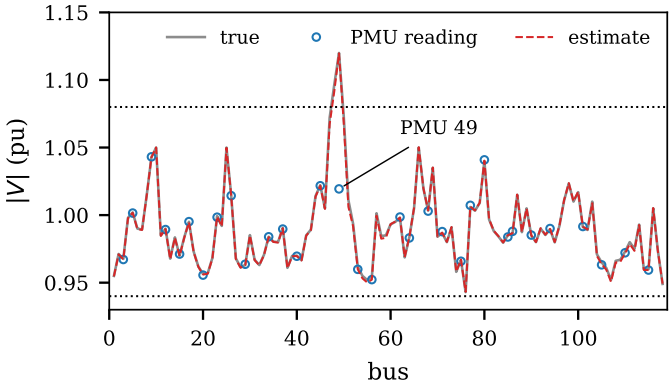


Fig. 14. Bus voltages in the set stamped 4.1 s, simple FDI on bus 49.

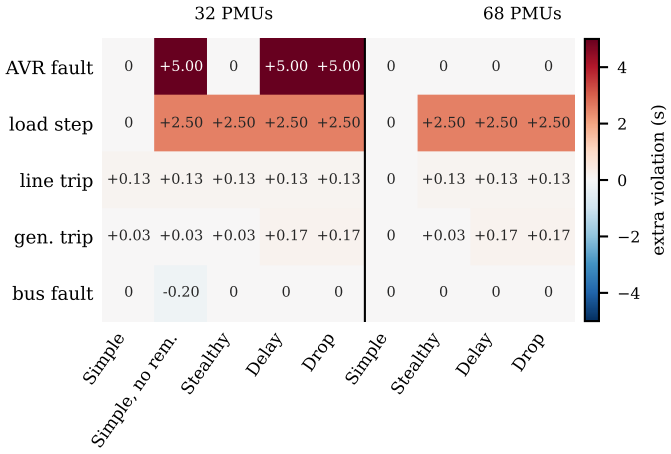


Fig. 15. Extra violation caused by each attack, per fault type (s).

the grid stayed observable after removal and the second test passed ($J = 77$ against 434 in seed 1). The stealthy FDI now needed five PMUs and was again not detected, but the neighbours exposed the fault. Under delay and drop of all five PMUs that observe bus 49, bus 48 remained observed through other PMUs and was estimated at 1.096 pu, and the command was issued at 4.067 s. Every attack in the set left the violation at 0.5 s. Section VI-E shows that this protection depends on the neighbourhood of bus 49 and does not hold at most other locations.

Random loss. With 5% of frames lost on every link, only 20.5% of the 32-PMU sets were complete. Because the loss was spread over all PMUs, the frames of bus 49's PMUs arrived in most sets, and the violation was corrected as without loss. With 68 PMUs only 3.0% of the sets were complete, since a set needs all 68 frames, but 85.3% were still observable, against 20.5% with 32 PMUs.

D. Attacks Across Fault Types

Table VII applies every attack to every fault type, and Fig. 15 shows the extra violation that each attack caused compared with the attack-free control. All five seeds gave the same violation time in every cell, so each nonzero difference in Fig. 15 has the smallest p-value that an exact two-sided test can give with five runs per group, $p = 0.008$.

Load step at bus 53. The load step lowered bus 53 to 0.910 pu. Control raised the setpoint of generator 54 four times, one step per second, and cut the violation from 6.0 to 3.5 s. Bus 53 has only two neighbours, bus 52 and the generator bus 54, which holds its own voltage. Under minimum placement only PMU 53 observes it, and the redundant placement adds only PMU 52. The simple FDI was detected and removed, and the control acted as without attack. Every other attack defeated the control under both placements. The stealthy FDI hid the voltage with no alarm, and delay or drop removed the only frames that showed it. In each case the violation lasted 6.0 s, the same as without control. Redundant placement cannot help a bus whose degree limits how many PMUs can observe it.

Line trip 76–77 and generator trip at bus 80. Both faults depressed bus 76, which has no PMU under minimum placement and is observed through the current that PMU 77 measures. The attacks therefore had to act on PMU 77, and under redundant placement also on PMU 118. Bus 76 has one more neighbour, bus 118, which PMU 75 observes through its branch current. The depressed voltage of bus 118 kept the fault visible under every attack, and the first command was issued at 4.1 s in every run. The attacks cost one frame (33 ms) to five frames (167 ms) of extra violation. Under minimum placement the simple FDI on a current channel raised no alarm, because that channel was the only measurement of bus 76. Under redundant placement it was detected in every set and removed, and the attack had no effect.

TABLE VII
VIOLATION TIME (S) FOR EVERY FAULT TYPE AND ATTACK, IEEE 118-BUS SYSTEM. MEAN ± STANDARD DEVIATION OVER FIVE SEEDS; A SINGLE VALUE MEANS ALL SEEDS AGREED

	Baselines		32 PMUs					68 PMUs			
Fault	No ctrl	Ctrl	Simple	Simple, no rem.	Stealthy	Delay	Drop	Simple	Stealthy	Delay	Drop
AVR fault, gen. 49	6.00	0.50	0.50	5.50	0.50	5.50	5.50	0.50	0.50	0.50	0.50
Load step +200%, bus 53	6.00	3.50	3.50	6.00	6.00	6.00	6.00	3.50	6.00	6.00	6.00
Line trip 76–77	5.97	2.50	2.63	2.63	2.63	2.63	2.63	2.50	2.63	2.63	2.63
Generator trip, bus 80	2.80	1.63	1.67	1.67	1.67	1.80	1.80	1.63	1.67	1.80	1.80
Bus fault 49, 0.1 s	2.03	1.60	1.60	1.40	1.60	1.60	1.60	1.60	1.60	1.60	1.60

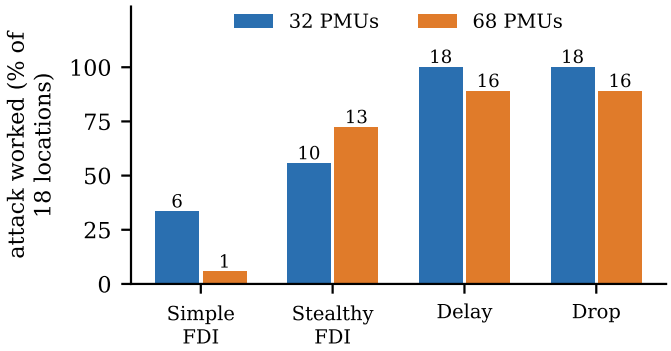


Fig. 16. Share of the 18 locations where each attack extended the violation.

Bus fault at bus 49. The three-phase fault drove bus 49 to zero for 0.1 s and depressed the whole area. No attack on the PMUs around bus 49 could hide it, and no attack extended the violation. The fault exposed a different weakness. During the fault the control center saw voltages far below the limit and issued 34 to 35 commands, which raised generator setpoints just before the fault cleared. After clearing, these commands caused over-voltage at bus 66 for 41 frames, against 35 frames without control. Detection without removal held most of these commands and gave the shortest violation, 1.4 s against 1.6 s with removal. A voltage controller that acts on single frames needs a time filter before it reacts to faults that clear on their own.

E. Fault and Attack Locations

The results above concern one target bus per fault. Table VIII and Fig. 16 repeat the attacks at 20 more locations: AVR faults at 16 generator buses with degree 1 to 8, and load steps of +200% at load buses 21, 52, 95 and 118. Each location ran with every attack under both placements, with seed 1, since the seeds did not change any outcome above. At buses 21 and 95 the commands of the control center did not clear the under-voltage within the run, so the violation lasted 6.0 s with and without control. These two locations say nothing about the attacks and are left out of the counts below, which therefore cover 18 locations.

Delay and drop extended the violation at all 18 locations with 32 PMUs and at 16 with 68 PMUs. They failed only after the AVR faults at generators 8 and 77, where the fault also raised a neighbouring bus (bus 9 to 1.099 pu, buses 78 and 79 to 1.105 and 1.085 pu) that PMUs outside the attacked set still observed. This is the same mechanism as at bus 49, where bus 48 exposed the fault. The result of Section VI-C for bus 49 is therefore an exception.

The *stealthy FDI* extended the violation at 10 locations with 32 PMUs and at 13 with 68 PMUs. Of the eight locations where it did not work with 32 PMUs, two (generators 12 and 77) were again exposed by a neighbour. At the other six the attacker did not hold a PMU at the target bus, so it used a fixed offset computed from the operating point it expected. The offset overshot, the estimate fell below the lower limit, and the controller raised the setpoint of the faulted generator. In the dispatch mode that absolute setpoint replaced the faulty reference by accident. With 68 PMUs the attacker held a PMU at the target in 14 of the 18 locations, against 7 with 32 PMUs, could follow the grid, and succeeded more often.

The *simple FDI* worked at 6 locations with 32 PMUs. At three of them (generator 10 and load buses 52 and 118) it changed a critical channel and raised no alarm. At the other three (generators 25, 59 and 90) the test caught it in every set and the suspect PMU was removed, but removal left the faulted bus unobservable. The pseudo-measurements then held its voltage at the value before the fault, and the fault stayed hidden for the whole attack. Detection and removal alone are therefore not enough when removal costs observability. With 68 PMUs the simple FDI worked at one location only.

In the telemetry mode, the violation lasted 2.5 s at all 17 AVR locations, against 0.5 s in the dispatch mode, for the reason given in Section VI-B. Across locations, one rule explains the outcomes: an attack that hides or shifts the measurements of the faulted bus succeeds unless the fault also drives a bus out of limits that unattacked PMUs observe. Redundant placement by (3) changed the outcome of delay and drop at only two of the 18 locations, because the attacker in this study takes every PMU that observes the target, however many there are.

Seeds. Table IX repeats ten of these locations, those where a neighbour, removal or a critical channel decided the outcome, with seeds 2 to 5. In 58 of the 60 attack and placement cells the attack either worked in all five seeds or in none. The two exceptions were both simple injections. At generator 90 with 32 PMUs the attack worked in three seeds out of five. At load bus 118 with 68 PMUs it worked in two seeds, where the load variation kept the voltage close to the limit when the PMU was removed. At load bus 52 the attack-free control itself varied between 1.5 and 3.5 s with the seed, so the extra violation of the attacks varied too, but every attack worked in

TABLE VIII
VIOLATION TIME (S) AT 20 FAULT LOCATIONS, IEEE 118-BUS SYSTEM, SEED 1. ATTACKS TARGET THE FAULTED BUS FROM 4 TO 9 S. BOLD: THE ATTACK EXTENDED THE VIOLATION BEYOND THE ATTACK-FREE CONTROL

		PMUs seeing bus		No attack			32 PMUs				68 PMUs			
Fault	Degree	32	68	No ctrl	Dispatch	Telemetry	Simple	Stealthy	Delay	Drop	Simple	Stealthy	Delay	Drop
AVR, gen. 10	1	1	2	6.0	0.5	2.5	**5.5**	**5.5**	**5.5**	**5.5**	0.5	**5.5**	**5.5**	**5.5**
AVR, gen. 87	1	1	2	6.0	0.5	2.5	0.5	0.5	**5.5**	**5.5**	0.5	**5.5**	**5.5**	**5.5**
AVR, gen. 26	2	1	2	6.0	0.5	2.5	0.5	**5.5**	**5.5**	**5.5**	0.5	**5.5**	**5.5**	**5.5**
AVR, gen. 90	2	1	2	6.0	0.5	2.5	**6.0**	**5.5**	**5.5**	**5.5**	0.5	0.5	**5.5**	**5.5**
AVR, gen. 8	3	2	3	6.0	0.5	2.5	0.5	0.5	**5.5**	**5.5**	0.5	0.5	0.5	0.5
AVR, gen. 25	3	2	2	6.0	0.5	2.5	**5.5**	**5.5**	**5.5**	**5.5**	0.5	**5.5**	**5.5**	**5.5**
AVR, gen. 40	4	2	3	6.0	0.5	2.5	0.5	**5.5**	**5.5**	**5.5**	0.5	**5.5**	**5.5**	**5.5**
AVR, gen. 66	4	2	3	6.0	0.5	2.5	0.5	**5.5**	**5.5**	**5.5**	0.5	**5.5**	**5.5**	**5.5**
AVR, gen. 89	4	2	3	6.0	0.5	2.5	0.5	0.5	**5.5**	**5.5**	0.5	**5.5**	**5.5**	**5.5**
AVR, gen. 15	5	2	3	6.0	0.5	2.5	0.5	**5.5**	**5.5**	**5.5**	0.5	**5.5**	**5.5**	**5.5**
AVR, gen. 54	5	3	4	6.0	0.5	2.5	0.5	0.5	**5.5**	**5.5**	0.5	0.5	**5.5**	**5.5**
AVR, gen. 59	6	1	3	6.0	0.5	2.5	**5.5**	0.5	**5.5**	**5.5**	0.5	**5.5**	**5.5**	**5.5**
AVR, gen. 77	6	3	3	6.0	0.5	2.5	0.5	0.5	**5.5**	**5.5**	0.5	0.5	0.5	0.5
AVR, gen. 12	7	2	4	6.0	0.5	2.5	0.5	0.5	**5.5**	**5.5**	0.5	0.5	**5.5**	**5.5**
AVR, gen. 80	7	2	4	6.0	0.5	2.5	0.5	**5.5**	**5.5**	**5.5**	0.5	**5.5**	**5.5**	**5.5**
AVR, gen. 100	8	2	4	6.0	0.5	2.5	0.5	0.5	**5.5**	**5.5**	0.5	**5.5**	**5.5**	**5.5**
Load step, bus 21	2	1	2	6.0	6.0	–	6.0	6.0	6.0	6.0	6.0	6.0	6.0	6.0
Load step, bus 52	2	1	3	6.0	3.0	–	**5.5**	**5.5**	**5.5**	**5.5**	3.0	**5.5**	**5.5**	**5.5**
Load step, bus 95	2	1	2	6.0	6.0	–	6.0	6.0	6.0	6.0	6.0	6.0	6.0	6.0
Load step, bus 118	2	1	2	6.0	1.5	–	**6.0**	**6.0**	**6.0**	**6.0**	**2.0**	**6.0**	**6.0**	**6.0**

TABLE IX
EXTRA VIOLATION (S) OVER THE ATTACK-FREE CONTROL AT TEN LOCATIONS, SEEDS 1–5 (MEAN; RANGE IN BRACKETS WHEN THE SEEDS DIFFER)

	Simple FDI		Stealthy FDI		Delay	
Fault	32	68	32	68	32	68
AVR 8	0.0	0.0	0.0	0.0	5.0	0.0
AVR 10	5.0	0.0	5.0	5.0	5.0	5.0
AVR 12	0.0	0.0	0.0	0.0	5.0	5.0
AVR 25	5.0	0.0	5.0	5.0	5.0	5.0
AVR 54	0.0	0.0	0.0	0.0	5.0	5.0
AVR 59	5.0	0.0	0.0	5.0	5.0	5.0
AVR 77	0.0	0.0	0.0	0.0	5.0	0.0
AVR 90	3.3 [0.0, 5.5]	0.0	5.0	0.0	5.0	5.0
Load 52	3.1 [2.0, 4.0]	0.0	3.1 [2.0, 4.0]	3.1 [2.0, 4.0]	3.1 [2.0, 4.0]	3.1 [2.0, 4.0]
Load 118	4.5	1.9 [0.0, 4.5]	4.5	4.5	4.5	4.5

every seed. The location results therefore do not depend on the seed, except for small injections near the threshold of the test.

F. Summary by Attack Type

Table X brings the results of all attack types together. The *simple FDI* is the only attack that the chi-square test catches. It still worked where it changed a critical channel, which raises no alarm, and where removal of the caught PMU left the faulted bus unobservable. Without removal the alarm blocked the control, and the attack became one of the most harmful. The *stealthy FDI* never raised an alarm and hid the fault wherever no observed neighbour showed it and its offset matched the grid. *Delay* and *drop* never raised an alarm either, because they leave the set incomplete, and a set that is not observable is not tested. They had identical effects in every case, since a frame that misses the wait window is as useless as a lost one, and they were the most reliable attacks, working at almost every location with both placements. *Random frame loss* of 5% on every link, tested with the AVR fault, reduced the share of complete sets to 20.5% with 32 PMUs and 3.0% with 68 PMUs, but it did not hide the fault, because it does not single out the PMUs that matter.

G. Dynamic Response

The three dynamic faults show what the dynamic mode adds. Fig. 17 shows the three-phase fault at bus 49. The voltage of bus 49 fell to zero from 4.03 to 4.10 s and that of bus 66 to 0.57 pu. After clearing, the excitation systems drove bus 66 above the 1.08 pu limit for 35 frames (1.2 s). With control, the commands issued during the fault raised the peak from 1.094 to 1.096 pu and kept it above the limit for six more frames, as discussed in Section VI-D.

Figs. 18 and 19 show the trip of the generator at bus 80. Bus 76 dropped to 0.926 pu. Without control it recovered slowly and crossed back above 0.94 pu at 6.83 s. With control, two setpoint commands to generator 76 brought it back at 5.67 s and to 0.957 pu by the end of the run. The loss of generation lowered the centre-of-inertia frequency to a minimum of 59.454 Hz at 7.07 s, and the lowest machine frequency to 59.431 Hz, after which the governors began to restore it. The voltage control changed the frequency minimum by less than

TABLE X
SUMMARY OF THE FOUR ATTACK TYPES AND RANDOM FRAME LOSS

Attack	What the attacker changes	Chi-square alarm	Worked at, of 18 locations		Why it failed elsewhere	Five fault types at fixed buses (Table VII)
			32 PMUs	68 PMUs		
Simple FDI	One channel of a PMU at or next to the target bus	Yes, unless the channel is critical	6	1	Removal kept the bus observable and the estimate correct	Up to 0.13 s extra; 5.0 s without removal
Stealthy FDI	Every channel that depends on the target bus	Never	10	13	A neighbour exposed the fault, or a fixed offset overshot	Hid the load step (2.5 s)
Delay, 100 ms	Holds those PMUs' frames past the wait window	Never (set not testable)	18	16	A neighbour exposed the fault	Hid the AVR fault (32 PMUs) and the load step
Drop	Discards those PMUs' frames	Never (set not testable)	18	16	A neighbour exposed the fault	Same as delay
Random loss, 5%	Every link, no attacker	Never	–	–	Not aimed at the faulted bus	No effect (AVR fault only)

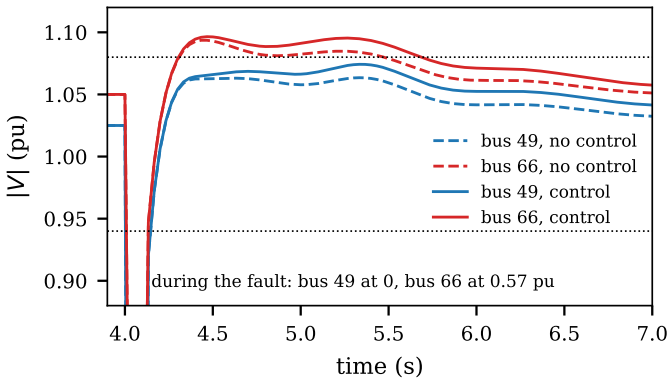


Fig. 17. Bus voltages around the three-phase fault at bus 49 (seed 1).

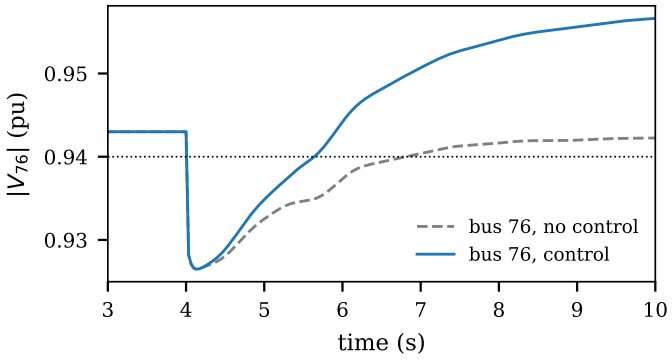


Fig. 18. Voltage of bus 76 after the trip of generator 80 (seed 1).

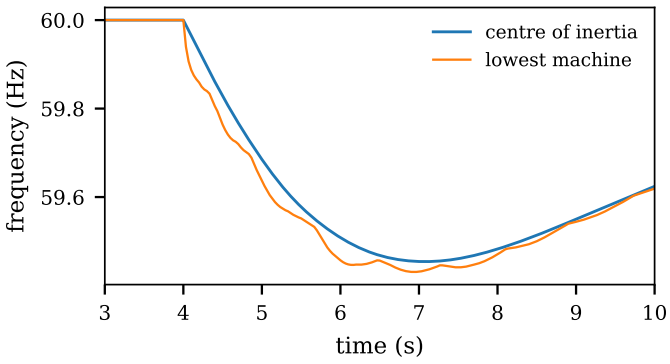


Fig. 19. Frequency after the trip of generator 80, with control (seed 1).

0.004 Hz. The control center in this study acts on voltage only. A frequency-based controller could use the same frames.

H. Detector Calibration With Matched Noise

Table XI and Fig. 20 measure the chi-square test without a fault. With the default $\sigma = 0.002$ the statistic averaged 25.5 with 32 PMUs, a quarter of its 102 degrees of freedom, and no set raised a false alarm. The price was sensitivity: an offset of 0.02 pu on the voltage channel of PMU 49 was detected in 4% of the sets and only 0.05 pu was detected reliably. With the matched $\sigma = 0.001$ the statistic averaged 101.5, close to its 102 degrees of freedom, and 22 of 2980 attack-free sets, 0.74%, raised a false alarm, as expected for a test at the 1% level. Detection rose to 20% of the sets at 0.005 pu and 98.7% at 0.01 pu with 32 PMUs. With 68 PMUs it was lower at the same offset, 74.9% at 0.01 pu, because the threshold grows with the degrees of freedom while the falsified channel stays one among 315. When the test raised an alarm during an attack, the normalized residual pointed first to PMU 49 in at least 99.5% of the sets, and no set stayed in alarm after removal. Each false alarm, in contrast, removed a correct PMU.

With the matched σ, the AVR-fault attacks at bus 49 gave the same violation times as with the default, 0.5 s for the control without attack and for both injections with both placements in every seed. The simple FDI raised an alarm in 36 to 48 sets with 32 PMUs and 22 to 28 with 68 PMUs, instead of 15, because the falsified voltage stayed detectable after the command had corrected the fault. The stealthy FDI raised 2 to 6 alarms per run, about as many as the 1 to 4 false alarms of the attack-free control runs, so it remained invisible to the test. The conservative default weights therefore did not change the outcomes reported above, but they hide injections smaller than about 0.05 pu, which matched weights

TABLE XI
BAD DATA DETECTION WITHOUT A FAULT, SIMPLE FDI ON BUS 49 FROM 2 TO 9 S, FIVE SEEDS PER CELL. ESTIMATOR σ = 0.002 (DEFAULT) OR 0.001 (MATCHED TO THE SIMULATED NOISE)

σ	PMUs	Mean J	False alarms	Detected sets (%) for an offset of					PMU 49 removed
		no attack	(% of sets)	0.002 pu	0.005 pu	0.010 pu	0.020 pu	0.050 pu	first (%)
0.002	32	25.5	0.00	0.0	0.0	0.0	4.0	99.5	100.0
0.002	68	98.4	0.00	0.0	0.0	0.0	0.0	99.5	100.0
0.001	32	101.5	0.81	1.5	20.1	98.7	99.5	99.5	99.7
0.001	68	391.3	0.67	1.0	5.7	74.9	99.5	99.5	99.5

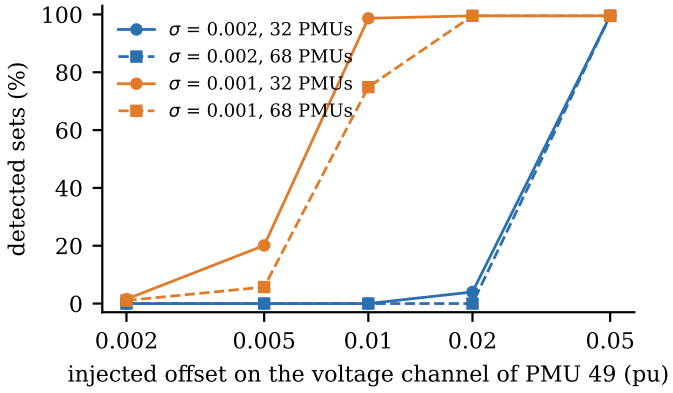


Fig. 20. Share of PDC sets in which the test detected a simple FDI, by offset size.

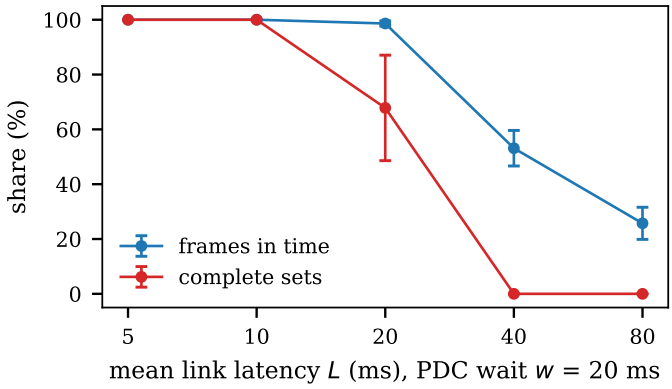


Fig. 21. Frames in time and complete sets against mean link latency (w = 20 ms).

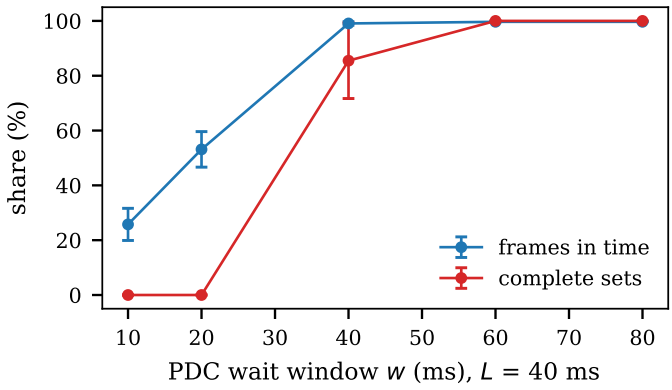


Fig. 22. Frames in time and complete sets against PDC wait window (L = 40 ms).

would catch at a cost of about one false alarm in a hundred sets.

I. Network Latency and the PDC Wait Window

Table XII and Fig. 21 repeat the controlled run without attack at other mean latencies. Because each link delay is drawn between $0.5L$ and $1.5L$ by (1), the spread of delays grows with L, while the wait window is measured from the first frame of a time stamp. Once the spread exceeds w, the slow links miss their sets. At $L = 20$ ms the share of complete sets fell to $67.8 \pm 26.9\%$ and depended strongly on the seed, between 28% and 100%, while 98.6% of the frames still arrived in time. At $L = 40$ ms no set was complete or observable in any seed, and whether the fault was seen depended on the delays drawn for PMUs 45 and 49. In seed 1, PMU 49's link had 32 ms against a fastest link of 21 ms, its frames fit in the window, and the command was issued at 4.100 s. In seed 3 its link had 49.5 ms against 20.1 ms, its frames were always late, and the violation lasted to the end of the run. Over ten seeds the fault went unseen in six, and the violation averaged 3.8 s with no attack at all. At $L = 80$ ms the fault went unseen in nine seeds out of ten. The mean absolute error of the estimate rose from 2.8×10^{-4} pu to 9.1×10^{-3} pu at 40 ms and 1.6×10^{-2} pu at 80 ms, because the unobserved buses were held near earlier values.

Fig. 22 shows the remedy at $L = 40$ ms. A wait window of 10 ms made matters worse, with the fault unseen in nine seeds. A window of 40 ms, equal to the expected spread L, restored $85.5 \pm 19.3\%$ complete sets and corrected the fault in every seed, and 60 ms restored all sets. The price is latency. The PDC released sets 62 ms after their time stamp with $w \geq 40$ ms, against 18 ms at $L = 10$ ms. Beyond 60 ms, a longer window brought nothing, because every frame had already arrived. The wait window therefore has to follow the spread of the link delays, and their mean is a poor guide.

J. Ablation and Statistical Significance

Table XIII tests the main comparisons. Removing the bad PMU, adding redundant PMUs against delay and drop, and closing the voltage loop each changed the violation time by 5.0 to 5.5 s with $p = 1.1 \times 10^{-5}$, the smallest value an exact two-sided test can give with ten runs per group, since all twenty runs fell on the expected side. These small values say that the random factors of the seeds (noise, link delays,

TABLE XII
EFFECT OF MEAN LINK LATENCY L AND PDC WAIT WINDOW w, IEEE 118-BUS SYSTEM, MINIMUM PLACEMENT, AVR FAULT, NO ATTACK. MEAN ± STANDARD DEVIATION OVER TEN SEEDS

L (ms)	w (ms)	Frames in time (%)	Complete sets (%)	Observable sets (%)	Latency p50 (ms)	Latency p95 (ms)	PDC release delay (ms)	Estimation error (10^{-3} pu)	Violation time (s)
5	20	100.0	100.0	100.0	7.8 ± 0.4	10.2 ± 0.3	10.5 ± 0.1	0.28 ± 0.00	0.5
10	20	100.0	100.0	100.0	12.8 ± 1.0	17.3 ± 0.3	17.9 ± 0.2	0.28 ± 0.00	0.5
20	20	98.6 ± 1.3	67.8 ± 26.9	67.8 ± 26.9	22.8 ± 1.8	31.5 ± 0.7	32.5 ± 0.2	0.29 ± 0.02	0.5
40	20	53.1 ± 9.1	0.0	0.0	34.2 ± 2.2	42.7 ± 1.7	44.0 ± 1.8	9.07 ± 2.67	3.8 ± 2.8
80	20	25.7 ± 8.2	0.0	0.0	53.9 ± 4.6	62.1 ± 5.0	65.4 ± 3.5	16.33 ± 2.00	5.5 ± 1.7
40	10	25.8 ± 8.2	0.0	0.0	28.3 ± 2.3	32.5 ± 2.5	34.0 ± 1.8	15.99 ± 2.09	5.5 ± 1.7
40	40	99.1 ± 0.9	85.5 ± 19.3	85.5 ± 19.3	43.2 ± 3.7	60.7 ± 1.1	62.2 ± 0.4	0.29 ± 0.01	0.5
40	60	99.7	100.0	100.0	43.3 ± 3.7	60.9 ± 1.1	62.2 ± 0.4	0.28 ± 0.00	0.5
40	80	99.7	100.0	100.0	43.3 ± 3.7	60.9 ± 1.1	62.2 ± 0.4	0.28 ± 0.00	0.5

TABLE XIII
STATISTICAL TESTS BETWEEN CONFIGURATIONS, TEN SEEDS EACH. DIFFERENCE OF MEANS (B − A) WITH 95% CONFIDENCE INTERVAL AND TWO-SIDED EXACT PERMUTATION TEST

Comparison (A vs. B)	Mean A	Mean B − A	p
Control (A) vs. no control (B)	0.5 s	+5.5 s	1.1×10^{-5}
Simple FDI, 32 PMUs: removal (A) vs. no removal (B)	0.5 s	+5.0 s	1.1×10^{-5}
No attack (A) vs. simple FDI with removal (B)	0.5 s	+0.0 s	1
No attack (A) vs. stealthy FDI (B), 32 PMUs	0.5 s	+0.0 s	1
Delay: 68 PMUs (A) vs. 32 PMUs (B)	0.5 s	+5.0 s	1.1×10^{-5}
Drop: 68 PMUs (A) vs. 32 PMUs (B)	0.5 s	+5.0 s	1.1×10^{-5}
5% loss, observable sets: 68 PMUs (A) vs. 32 PMUs (B)	85.3 %	-64.8 [-66.4, -63.1] %	1.1×10^{-5}
Complete sets: L = 10 ms (A) vs. 20 ms (B)	100.0 %	-32.2 [-50.0, -14.3] %	0.003
Complete sets at L = 40 ms: w = 40 ms (A) vs. 20 ms (B)	85.5 %	-85.5 [-98.3, -72.7] %	1.1×10^{-5}
Violation at L = 40 ms: w = 40 ms (A) vs. 10 ms (B)	0.5 s	+5.0 [+3.8, +6.1] s	1.2×10^{-4}
Violation: L = 10 ms (A) vs. 80 ms (B)	0.5 s	+5.0 [+3.8, +6.1] s	1.2×10^{-4}

loss and load variation) cannot explain the differences. They say nothing about other faults, locations or grids, which the location campaign of Section VI-E addresses. The simple FDI with removal and the stealthy FDI did not change the violation time compared with the attack-free run in any seed, so the test cannot reject equality ($p = 1$). For the network, the drop in complete sets from 10 to 20 ms ($p = 0.003$), the gain from a 40 ms instead of a 20 ms window at $L = 40$ ms, and the longer violations at 80 ms latency or with a 10 ms window ($p = 1.2 \times 10^{-4}$) were all significant at the 1% level. Redundant placement under 5% loss raised the observable sets by 64.8 percentage points (95% confidence interval 63.1 to 66.4).

K. Computing Time

Table XIV gives the GridPACK time per grid step and the wall-clock time of a 10 s experiment with one, two and four ranks. One- and two-rank runs shared the cluster with

TABLE XIV
COMPUTING TIME PER EXPERIMENT WITH 1, 2 AND 4 GRIDPACK MPI RANKS, BASE SCENARIO, FIVE SEEDS EACH (MEAN ± STANDARD DEVIATION)

Ranks	Solve time (ms)	Grid step (ms)	Wall time (s)
1	9.6 ± 1.6	17.5 ± 4.2	4.8 ± 0.4
2	32.6 ± 10.5	52.1 ± 18.7	6.1 ± 0.6
4	90.5 ± 19.5	136.0 ± 15.1	7.3 ± 0.3

TABLE XV
CONTROL-CENTER COMPUTING TIME PER PDC SET, IEEE 118-BUS SYSTEM

PMUs	Set	Median (ms)	p95 (ms)	Max (ms)
32	clean	0.91	1.50	3.90
32	FDI, PMU 49 removed	19.39	29.58	37.32
68	clean	0.85	1.53	2.30
68	FDI, PMU 49 removed	37.60	51.46	56.84

a second experiment, while four-rank runs used all four MPI slots, so the queue ran them alone. A 118-bus power flow takes about 10 ms on one rank, so every additional rank adds more communication than it removes work, and four ranks split over the two compute nodes, which exchange MPI messages over TCP, needed 90 ms per solve. For this grid one rank is the right choice, and the cluster pays off by running many experiments at once. The runs of the study took between 4.0 and 9.0 s each, except the first two after the cluster started, which took 20 s.

Table XV gives the control center's time per set, measured by a benchmark of the estimator on the IEEE 118-bus system with 200 noisy sets per case. A clean set took a median of 0.9 ms with either placement. When an alarm triggered the normalized-residual test, which needs one solve per channel, and a second estimate, the median rose to 19 ms with 32 PMUs and 38 ms with 68 PMUs, with a maximum of 57 ms. These times do not affect the simulated time of a run, but they matter for a real-time version, because a frame period is 33.3 ms.

L. Comparison With Prior Work

Direct numerical comparison with the platforms of Table I is limited, because they study other systems, events and metrics. Where the quantities match, the results agree. The median PMU-to-PDC latencies of 12.8 ms at $L = 10$ ms and 22.8 ms

at $L = 20$ ms bracket the mean of about 20 ms that Morato *et al.* [30] found for PMU traffic over 5G. Our earlier IEEE 14-bus prototype reported latencies rising from 10 to about 100 ms under denial of service but could not show the effect on the grid [24]. The present study shows that a delay of that order hides an over-voltage for 5.5 s under minimum placement. Zheng *et al.* [23] found that GridPACK under HELICS runs faster than real time for a 3000-bus system on four processes. Our single-rank runs simulated 10 s of a smaller system in about 5 s of wall-clock time on the three-node cluster, with the full network and control center in the loop. The value of the chi-square test against stealthy attacks matches [6], [34]: it did not react at all. The benefit of placement that keeps the grid observable without the attacked PMUs matches the analysis of [37], here shown in a closed loop with a simulated network.

Calibration of the test. With the default weights the chi-square statistic in normal operation averaged 25.5 with minimum placement, between 24.7 and 26.3 over the seeds, well below its 102 degrees of freedom. Each real component of a residual then has about a quarter of the variance that the test assumes, and $(2m - 2n)/4 \approx 26$. The test raised no false alarm in any of the runs with these weights. Section VI-H shows what the matched weights change.

VII. Discussion

A. Interpretation

The experiments separate the defenses by what they protect. The chi-square test with normalized-residual removal protects the estimate, and with it the control action, against a falsified channel, but only while enough PMUs remain after removal to observe the faulted bus and test the result again. Where removal cost observability, the pseudo-measurements carried the voltage from before the fault and the fault stayed hidden, so a control center should not act on an untested set as if it were a trusted estimate. Detection without removal protects the estimate but blocks the control, which after the AVR fault cost five seconds of over-voltage in every seed. That the stealthy injection passes the test is known [6]. The testbed shows its closed-loop consequence: the injection hid the fault wherever no observed neighbour showed it, and the learned detectors of [9], [10] are one way to catch it. Delay and drop were the most reliable attacks. Neither changes a value, and both turn a measurement into a missing one, which is consistent with the role of data dropout in time-synchronization attacks [11], [12].

The location campaign shows one rule behind these outcomes. A fault stays visible under attack only if it also drives a bus out of limits that PMUs outside the attacked set observe. This is close to the condition Khalafi *et al.* [37] derived for locating attacks, that the grid stays observable without the attacked PMUs, but it adds the fault: an AVR fault at generator 49 also raised bus 48, while most AVR faults raised only their own bus. Topological redundancy by (3) did not change this at 16 of 18 locations, because an attacker who can take every PMU observing a bus is not stopped by a second one. Protection must therefore come from authenticated or physically protected PMUs at critical buses, or from placement that covers the area a likely fault affects. The PDC results agree with the observation that wait times must be set from the latency of the slowest stream [5]. Finally, the controller is part of the attack surface. A loop that acts on every frame reacts to transients, and after the bus fault its own commands caused post-fault over-voltage. In the dispatch mode its absolute setpoints also removed a faulty AVR reference even when an attack pushed the estimate the wrong way, which flattered the defenses in six of the stealthy cases.

B. Limitations

The study has several limits that bound its conclusions. It uses one grid, the IEEE 118-bus system. The quasi-steady-state faults were placed at 22 locations, but each dynamic fault was studied at one location, so the attack results for dynamic faults rest on those neighbourhoods. The attacker is assumed to hold every PMU it needs, whatever the placement, so the study does not measure resilience against a limited attack budget. The network model gives every PMU its own point-to-point link with uniformly distributed delay, bounded jitter and independent loss. It does not model multihop routing, shared congestion, correlated failures or delay distributions measured on an operating synchrophasor network. The PMU frames are abstract, so the testbed does not yet show interoperability with IEEE C37.118.2 devices. The voltage controller is a heuristic: it chooses the nearest generator, moves its setpoint by a fixed step, does not model reactive power limits or the stability of the control loop, and has no time filter. After an AVR fault its first command overwrites the faulty reference in the dispatch mode, which is why that mode recovers faster than the telemetry mode. When removal leaves a set unobservable, the estimator completes it with pseudo-measurements and the set is not tested, so stale or corrupted information can still reach the controller. The attack campaigns use an estimator standard deviation twice the simulated one, which makes the chi-square test conservative. The matched-noise experiment of Section VI-H measures the effect for one bus and the simple injection, but the other attacks and locations were not repeated with matched weights. Finally, the testbed has not been compared with a real PMU and PDC installation or a hardware-in-the-loop setup, and the co-simulation is not paced to wall-clock time. The three cluster nodes shared one four-core server, so the MPI times include contention between the nodes and may differ on a cluster of separate servers.

C. Threats to Validity

Internal validity. The outcome depends on the time coupling. We derived the loop latency in (7) and checked it against the logged times of every command. The grid solution was checked against an independent Newton–Raphson solver. Each scenario changed one factor from the base, and all runs used the same software build.

Construct validity. The violation time is measured at grid steps, so in QSS mode differences smaller than 0.5 s are not visible. The attack campaigns use conservative estimator

weights. The matched-noise experiment shows that this hides small injections but did not change the outcomes at bus 49.

Statistical validity. The seeds vary noise, delays, loss and loads but not the fault or the attack, which are fixed by design. Many outcomes were therefore identical in all seeds. The tests show that the differences cannot be explained by these random factors, but they do not cover other fault locations, targets or attack sizes.

External validity. The experiments use one transmission network. The quasi-steady-state faults were evaluated at 22 locations, but each dynamic fault type was assessed at one location, and most additional locations were run with one random seed, with ten of them repeated for five seeds. The campaign shows that the outcome depends on the neighbourhood of the attacked bus and on the type of fault, so other locations, other grids and other attack sizes may change the numbers. The testbed can run those cases from the same scenario files.

VIII. Conclusion

This paper presented a remotely accessible co-simulation testbed in which GridPACK on a computing cluster, NS-3 with a PMU network and a PDC, and a control center with state estimation, bad data detection and voltage control run as HELICS federates, and in which several users start, follow and compare experiments from a browser. On the IEEE 118-bus system, 1087 runs covered five fault types in quasi-steady-state and dynamic simulation, 22 fault locations, four attack types, two PMU placements and sweeps of the network timing. The closed loop corrected each violation that the control center could see, but attacks that delayed, dropped or consistently shifted the measurements of the faulted bus hid the fault at most locations with either placement, and failed only where the fault also showed at a bus observed by unattacked PMUs. Removing a detected PMU helped only when the bus stayed observable, the PDC wait window had to cover the spread of link delays, and after a bus fault the voltage controller itself lengthened the violation. The main limitations are that the network model and PMU frames are abstract and that the controller is a simple heuristic whose absolute setpoints overwrite a faulty AVR reference. Future work will connect hardware PMUs through an IEEE C37.118.2 interface, pace the co-simulation to real time, add a second grid and shared network topologies, and add authenticated frames, learned detectors and a time-filtered controller.

Code and Data Availability

The testbed source code, installer, scenario files, extracted run data and the scripts that produce every figure and table in this paper are available at https://github.com/mazharulmd/cps-testbed-cluster.

Acknowledgment

The authors thank the Bangladesh Energy and Power Research Council for funding this work under grant EPRC/58-2018-007-01.

References

[1] A. Abur and A. Gómez Expósito, *Power System State Estimation: Theory and Implementation*. New York, NY, USA: Marcel Dekker, 2004.

[2] A. G. Phadke and J. S. Thorp, *Synchronized Phasor Measurements and Their Applications*. New York, NY, USA: Springer, 2008.

[3] "IEEE standard for synchrophasor measurements for power systems," IEEE Std C37.118.1-2011, 2011.

[4] "IEEE guide for phasor data concentrator requirements for power system protection, control, and monitoring," IEEE Std C37.244-2013, 2013.

[5] R. Pourramezan, H. Karimi, J. Mahseredjian, and M. Paolone, "Optimal coordination of phasor data concentrators in hierarchical synchrophasor networks," *IEEE Trans. Smart Grid*, vol. 12, no. 3, pp. 2402–2412, 2021.

[6] Y. Liu, P. Ning, and M. K. Reiter, "False data injection attacks against state estimation in electric power grids," in *Proc. 16th ACM Conf. Comput. Commun. Secur. (CCS)*, 2009, pp. 21–32.

[7] O. Kosut, L. Jia, R. J. Thomas, and L. Tong, "Malicious data attacks on the smart grid," *IEEE Trans. Smart Grid*, vol. 2, no. 4, pp. 645–658, 2011.

[8] R. B. Bobba, K. M. Rogers, Q. Wang, H. Khurana, K. Nahrstedt, and T. J. Overbye, "Detecting false data injection attacks on DC state estimation," in *Proc. 1st Workshop Secure Control Syst. (SCS)*, Stockholm, Sweden, 2010.

[9] C. Chen, Y. Wang, M. Cui, J. Zhao, W. Bi, Y. Chen, and X. Zhang, "Data-driven detection of stealthy false data injection attack against power system state estimation," *IEEE Trans. Ind. Informat.*, vol. 18, no. 12, pp. 8467–8476, 2022.

[10] A. Bhattacharjee, A. K. Mondal, A. Verma, S. Mishra, and T. K. Saha, "Deep latent space clustering for detection of stealthy false data injection attacks against AC state estimation in power systems," *IEEE Trans. Smart Grid*, vol. 14, no. 3, pp. 2338–2351, 2023.

[11] M. Zadsar *et al.*, "Preventing time-synchronization attacks on synchrophasor measurements of wide-area damping controllers," *IEEE Trans. Instrum. Meas.*, 2023.

[12] E. Shereen and G. Dán, "Network topology-aware mitigation of undetectable PMU time synchronization attacks," *IEEE Trans. Control Netw. Syst.*, 2024.

[13] K. Kumar, P. Saini, A. Prakash, S. K. Parida, K. A. Al Jaafari, H. H. Zeineldin, and E. F. El-Saadany, "Real-time cyber-physical co-simulation for resilient wide-area damping control against FDIAs and communication disruptions," *IEEE Trans. Ind. Appl.*, vol. 61, no. 6, 2025.

[14] T.-T. Nguyen, R. Kadavil, and H. Hooshyar, "A real-time cyber-physical simulation testbed for cybersecurity assessment of large-scale power systems," *IEEE Trans. Ind. Appl.*, vol. 60, no. 6, pp. 8329–8340, 2024.

[15] S. C. Das, T. Vu, and H. Ginn, "Scalable cyber-physical testbed for cybersecurity evaluation of synchrophasors in power systems," *IET Cyber-Phys. Syst.: Theory Appl.*, 2025.

[16] D. Mishchenko, I. Oleinikova, L. Erdődi, and B. R. Pokhrel, "Multidomain cyber-physical testbed for power system vulnerability assessment," *IEEE Access*, vol. 12, pp. 38 135–38 149, 2024.

[17] B. Palmintier, D. Krishnamurthy, P. Top, S. Smith, J. Daily, and J. Fuller, "Design of the HELICS high-performance transmission-distribution-communication-market co-simulation framework," in *Proc. Workshop Model. Simul. Cyber-Phys. Energy Syst. (MSCPES)*, Pittsburgh, PA, USA, 2017, pp. 1–6.

[18] T. D. Hardy, B. Palmintier, P. L. Top, D. Krishnamurthy, and J. C. Fuller, "HELICS: A co-simulation framework for scalable multi-domain modeling and analysis," *IEEE Access*, vol. 12, pp. 24 325–24 347, 2024.

[19] O. Bel, J. Kim, W. J. Hofer, M. Maharjan, B. Hyder, S. Purohit, and S. Niddodi, "Co-simulation framework for network attack generation and monitoring," *IEEE Access*, vol. 12, 2024.

[20] N. Thioub and M. Adda, "GridSecSim: A hybrid cyber-physical co-simulation platform integrating GridLAB-D and NS-3 for cyberattack simulation and real-time intrusion detection in smart grids," *IEEE Access*, vol. 14, 2026.

[21] S. C. Vallam Kondu, V. R. Motakatla, F. Ding, and J. Zhang, "A high-fidelity cyber-physical DER system testbed for analyzing DER monitoring and control under cyberattacks," in *Proc. IEEE Conf.*, 2025.

[22] M. Y. Masood, H. Rifà-Pous, M. Awais, A. Khawar, and M. Muzaffar, "Co-simulation of FDI and DoS attacks in IoT-enabled next-generation smart grids: A framework for resilience," in *Proc. 7th Int. Conf. Adv. Comput. Sci. (ICACS)*, 2026.

[23] L. Zheng, Y. Cui, S. Jin, and Y. Chen, "High-performance computing-based open-source power transmission and distribution grid co-

simulation," *IEEE Trans. Power Syst.*, vol. 39, no. 5, pp. 6144–6153, 2024.

[24] M. Islam, T. A. Khan, S. Zulfiker, A. S. M. J. Hasan, and H. A. Rahman, "Remotely accessible cyber-physical system testbed for power grid's security and reliability," in *Proc. 8th Int. Conf. Smart Grid Smart Cities (ICSGSC)*, 2024, pp. 436–441.

[25] B. Palmer, W. Perkins, Y. Chen, S. Jin, D. Callahan, K. Glass, R. Diao, M. Rice, S. Elbert, M. Vallem, and Z. Huang, "GridPACK: A framework for developing power grid simulations on high-performance computing platforms," *Int. J. High Perform. Comput. Appl.*, vol. 30, no. 2, pp. 223–240, 2016.

[26] G. F. Riley and T. R. Henderson, "The ns-3 network simulator," in *Modeling and Tools for Network Simulation*, K. Wehrle, M. Güneş, and J. Gross, Eds. Berlin, Germany: Springer, 2010, pp. 15–34.

[27] OpenJS Foundation, "Node-RED," [Online]. Available: https://nodered.org, 2024.

[28] "IEEE standard for synchrophasor data transfer for power systems," IEEE Std C37.118.2-2011, 2011.

[29] P. Castello, G. Gallus, P. A. Pegoraro, and S. Sulis, "Measurement platform for latency characterization of wide area monitoring, protection and control systems," *IEEE Trans. Instrum. Meas.*, vol. 73, p. 5500812, 2024.

[30] A. Morato, G. Frigo, and F. Tramarin, "5G-enabled PMU-based distributed measurement systems: Network infrastructure optimization and scalability analysis," *IEEE Trans. Instrum. Meas.*, vol. 73, 2024.

[31] E. Handschin, F. C. Schweppe, J. Kohlas, and A. Fiechter, "Bad data analysis for power system state estimation," *IEEE Trans. Power App. Syst.*, vol. PAS-94, no. 2, pp. 329–337, 1975.

[32] T. L. Baldwin, L. Mili, M. B. Boisen, and R. Adapa, "Power system observability with minimal phasor measurement placement," *IEEE Trans. Power Syst.*, vol. 8, no. 2, pp. 707–715, 1993.

[33] B. Gou, "Generalized integer linear programming formulation for optimal PMU placement," *IEEE Trans. Power Syst.*, vol. 23, no. 3, pp. 1099–1104, 2008.

[34] H. Yang, W. Zhang, C. Y. Chung, Z. Wang, W. Qiu, and Z. Liang, "AC false data injection attack based on robust tensor principle component analysis," *IEEE Trans. Ind. Informat.*, 2024.

[35] M. Zadsar, M. Ghafouri, A. Ameli, and B. Moussa, "Integrated deep learning-based detection and robust state observer mitigation of time-synchronization attacks: A WADC case study," *IEEE Trans. Smart Grid*, vol. 15, no. 5, pp. 5055–5071, 2024.

[36] K. P. Swain, A. Sharma, A. Karkare, S. Chakrabarti, E. Gryazina, and V. Terzija, "Network-level vulnerability assessment of synchrophasor measurement devices," *IEEE Access*, vol. 12, pp. 72 491–72 503, 2024.

[37] Z. S. Khalafi, M. Dehghani, A. Khalili, A. Sami, N. Vafamand, and T. Dragičević, "Intrusion detection, measurement correction, and attack localization of PMU networks," *IEEE Trans. Ind. Electron.*, vol. 69, no. 5, pp. 4697–4706, 2022.

[38] Z. Pourahmad, R.-A. Hooshmand, and M. Ataei, "Optimal placement of PMU and PDC in power systems by considering the vulnerabilities against cyber-attacks," *Elect. Eng.*, vol. 106, pp. 93–109, 2024.

[39] K. Hopkinson, X. Wang, R. Giovanini, J. Thorp, K. Birman, and D. Coury, "EPOCHS: A platform for agent-based electric power and communication simulation built from commercial off-the-shelf components," *IEEE Trans. Power Syst.*, vol. 21, no. 2, pp. 548–558, 2006.

[40] H. Lin, S. S. Veda, S. S. Shukla, L. Mili, and J. Thorp, "GECO: Global event-driven co-simulation framework for interconnected power system and communication network," *IEEE Trans. Smart Grid*, vol. 3, no. 3, pp. 1444–1456, 2012.

[41] K. Mets, J. A. Ojea, and C. Develder, "Combining power and communication network simulation for cost-effective smart grid analysis," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 3, pp. 1771–1796, 2014.

[42] S. Schütte, S. Scherfke, and M. Tröschel, "Mosaik: A framework for modular simulation of active components in smart grids," in *Proc. IEEE 1st Int. Workshop Smart Grid Model. Simul. (SGMS)*, 2011, pp. 55–60.

[43] D. Qiu, M. Liu, R. Zhang, T. Luo, A. Griffo, and X. Zhang, "Cyber-physical real-time digital simulation for cybersecurity analysis in microgrids," *IEEE Trans. Ind. Cyber-Phys. Syst.*, vol. 3, pp. 429–441, 2025.

[44] A. H. Dabashi, D. Qiu, X. Zhang, and G. Taylor, "Innovative testbed configurations and interfacing techniques for real-time digital simulation of cyber-physical energy and power systems," *Electronics*, vol. 15, p. 3705, 2026.

[45] A. Hahn, A. Ashok, S. Sridhar, and M. Govindarasu, "Cyber-physical security testbeds: Architecture, application, and evaluation for smart grid," *IEEE Trans. Smart Grid*, vol. 4, no. 2, pp. 847–855, 2013.

[46] A. Ashok, S. Krishnaswamy, and M. Govindarasu, "PowerCyber: A remotely accessible testbed for cyber physical security of the smart grid," in *Proc. IEEE PES Innov. Smart Grid Technol. Conf. (ISGT)*, Minneapolis, MN, USA, 2016, pp. 1–5.

[47] V. K. Singh, M. Govindarasu, D. Porschet, E. Shaffer, and M. Berman, "NEFTSec: Networked federation testbed for cyber-physical security of smart grid: Architecture, applications, and evaluation," *IET Cyber-Phys. Syst.: Theory Appl.*, vol. 7, no. 4, pp. 197–211, 2022.

[48] M. S. Abdelrahman, I. Kharchouf, T. L. Nguyen, and O. A. Mohammed, "A hybrid physical co-simulation smart grid testbed for testing and impact analysis of cyber-attacks on power systems: Framework and attack scenarios," *Energies*, vol. 16, no. 23, p. 7771, 2023.

[49] M. Chamana, R. Bhatta, K. Schmitt, R. Shrestha, and S. Bayne, "An integrated testbed for power system cyber-physical operations training," *Appl. Sci.*, vol. 13, no. 16, p. 9451, 2023.

[50] X. Wang *et al.*, "Improving cyber-physical-power system stability through hardware-in-loop co-simulation platform for real-time cyber attack analysis," *Front. Energy Res.*, vol. 12, 2024.

[51] B. Chng, B. Ng, M. M. Roomi, D. Mashima, and X. Lou, "CRaaS: Cloud-based smart grid cyber range for scalable cybersecurity experiments and training," in *Proc. IEEE Int. Conf. Commun., Control, Comput. Technol. Smart Grids (SmartGridComm)*, 2024, pp. 333–339.

[52] A. V. Jha *et al.*, "A graph-theoretic approach for modelling and resiliency analysis of synchrophasor communication networks," *Appl. Syst. Innov.*, vol. 6, no. 1, p. 7, 2023.

[53] S. Ramírez, "FastAPI," [Online]. Available: https://fastapi.tiangolo.com, 2024.

[54] G. Karypis and V. Kumar, "A parallel algorithm for multilevel graph partitioning and sparse matrix ordering," *J. Parallel Distrib. Comput.*, vol. 48, no. 1, pp. 71–95, 1998.

[55] P. R. Amestoy, I. S. Duff, J.-Y. L'Excellent, and J. Koster, "A fully asynchronous multifrontal solver using distributed dynamic scheduling," *SIAM J. Matrix Anal. Appl.*, vol. 23, no. 1, pp. 15–41, 2001.

[56] Message Passing Interface Forum, "MPI: A message-passing interface standard, version 4.0," 2021.

[57] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Math. Program. Comput.*, vol. 10, no. 1, pp. 119–142, 2018.

[58] R. D. Zimmerman, C. E. Murillo-Sánchez, and R. J. Thomas, "MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education," *IEEE Trans. Power Syst.*, vol. 26, no. 1, pp. 12–19, 2011.